

J.K.BATCH

EINFÜHRUNG IN DIE JAKARTA BATCH API

INHALT

- Was ist Batch API?
- Batch Architektur
- Job Ausführung
- Weitere Interfaces
- Job Specification Language (JSL)
- Job Scheduling
- Batches in Wildfly (mit CDI)
- Batch Job Design

WAS IST DIE BATCH API?

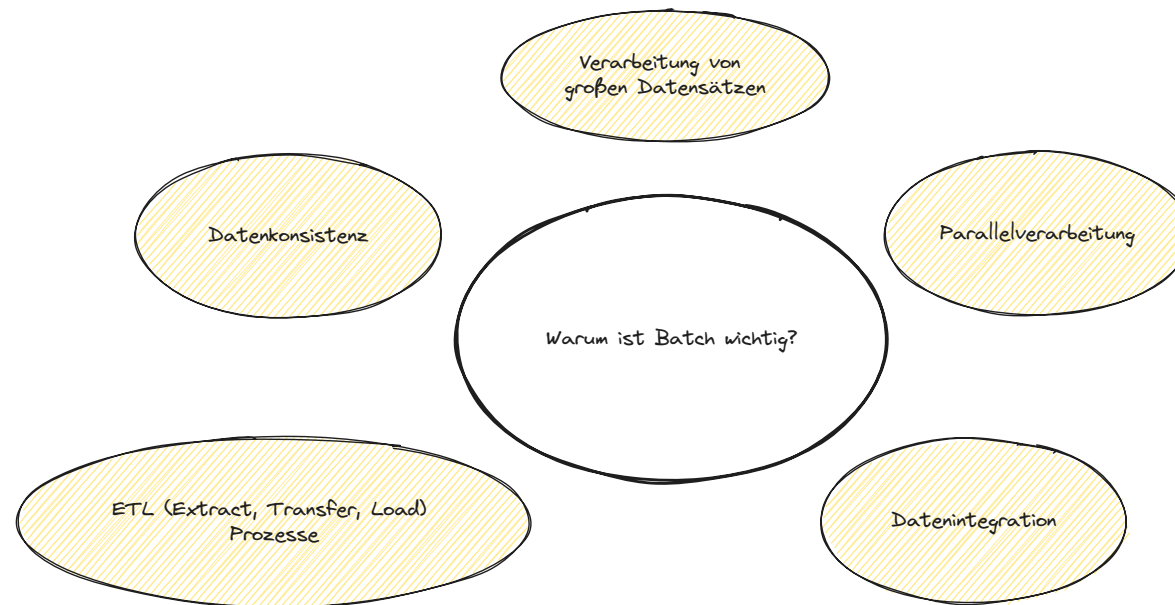
WAS IST BATCHVERARBEITUNG?

- Batchverarbeitung ist eine Software Lösung, die das Verarbeiten von großen Datenmengen in sogenannten "Chunks" oder "Batches" ermöglicht.
- Batches benötigen KEINE Benutzerinteraktion und können voll automatisiert ablaufen

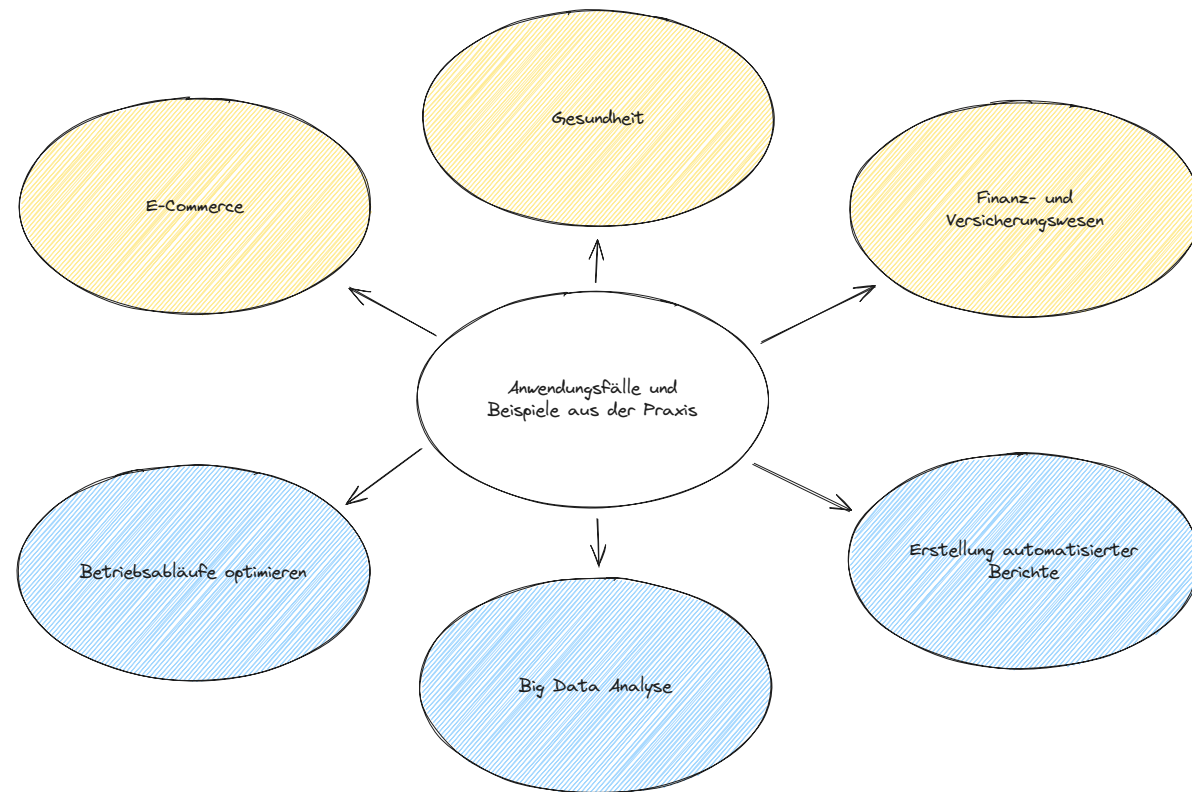
WAS IST DIE JAKARTA BATCH API?

- Jakarta EE, ehemals Java EE, standardisiert den Ansatz der Batchverarbeitung
- Jakarta Batch wird in Implementierungen wie JBeret von Wildfly, Batchee genutzt

WARUM IST BATCH API WICHTIG?

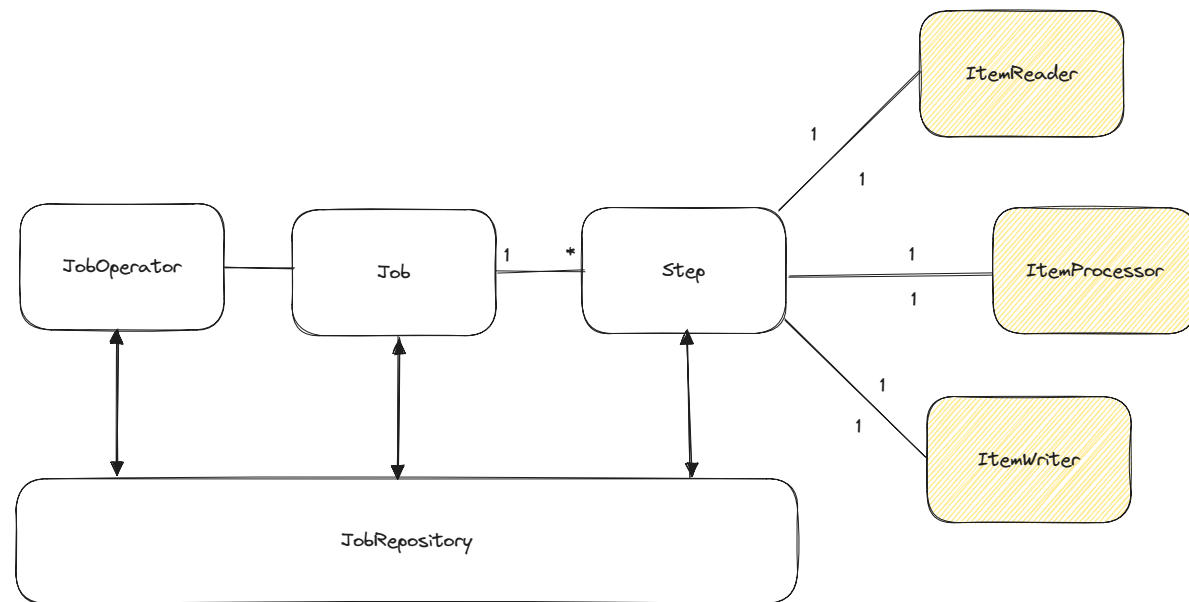


ANWENDUNGSFÄLLE AUS DER PRAXIS

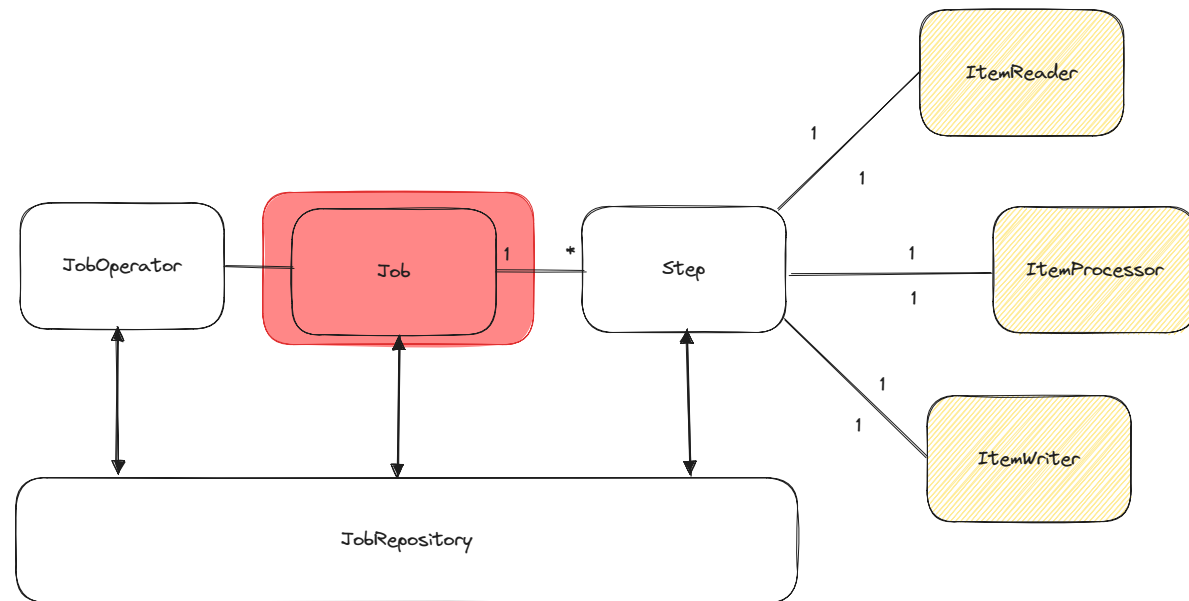


BATCH ARCHITEKTUR

ÜBERSICHT DER BATCH ARCHITEKTUR



WAS IST EIN JOB?

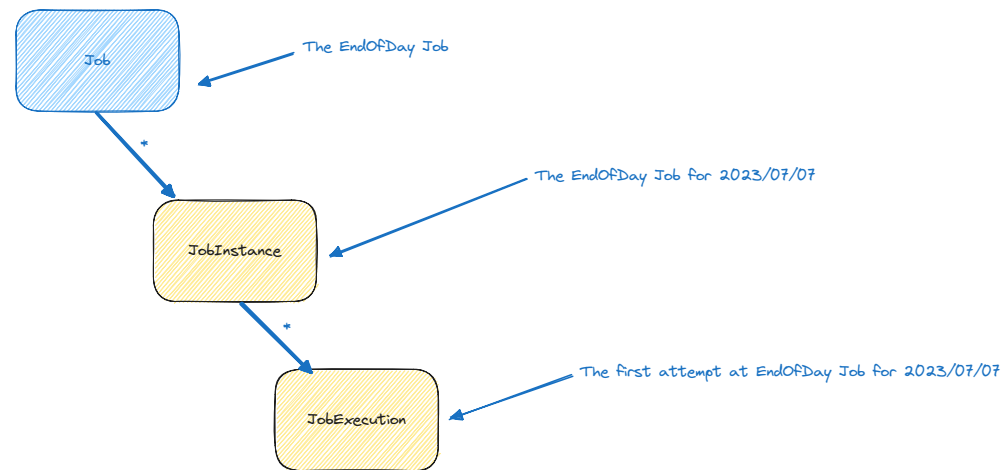


WAS IST EIN JOB?

- Zusammenfassung einzelner Teilschritte zur Durchführung der gewünschten fachlichen Aufgabe.
- Kann wiederanlauffähig sein
- Identifiziert bzw. referenziert durch eine eindeutige ID.
- Definition in Form einer XML Datei (<jobID>.xml).

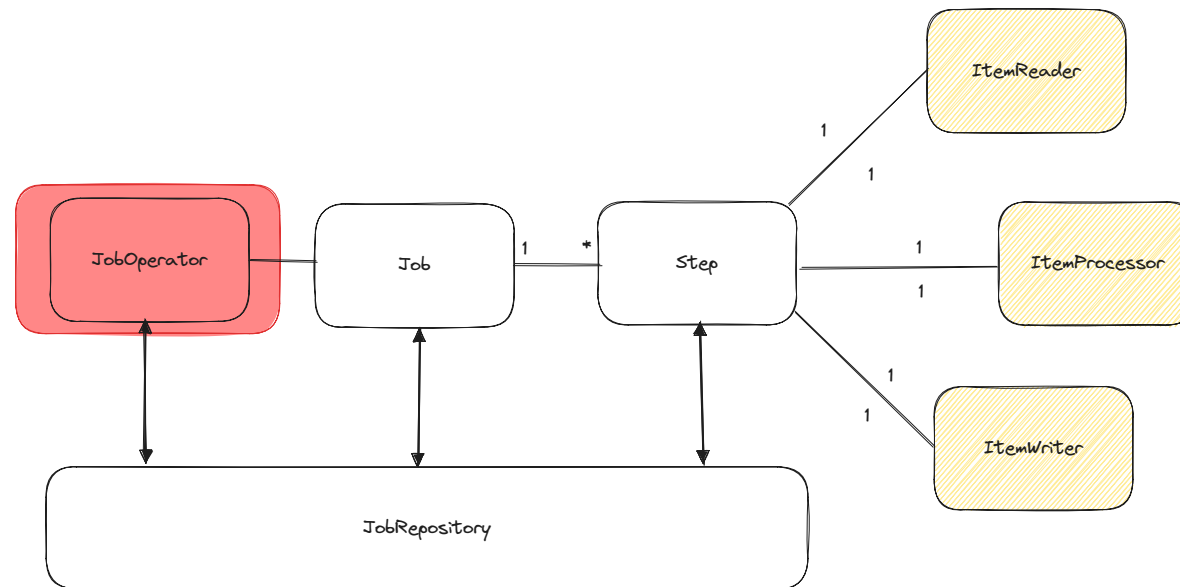
```
<job id="jobId" restartable="false">  
  <!-- Jobdefinition in Form einzelner Teilschritte -->  
</job>
```

JOBS ZUR LAUFZEIT



- ein Job besteht aus:
 - Job Instance: ist ein Identifikator, um mehrere Jobsauführungen mit verschiedenen Parametern zu verwalten
 - Job Execution: ein einzelner Versuch einen Job auszuführen

WAS IST EIN JOB OPERATOR?

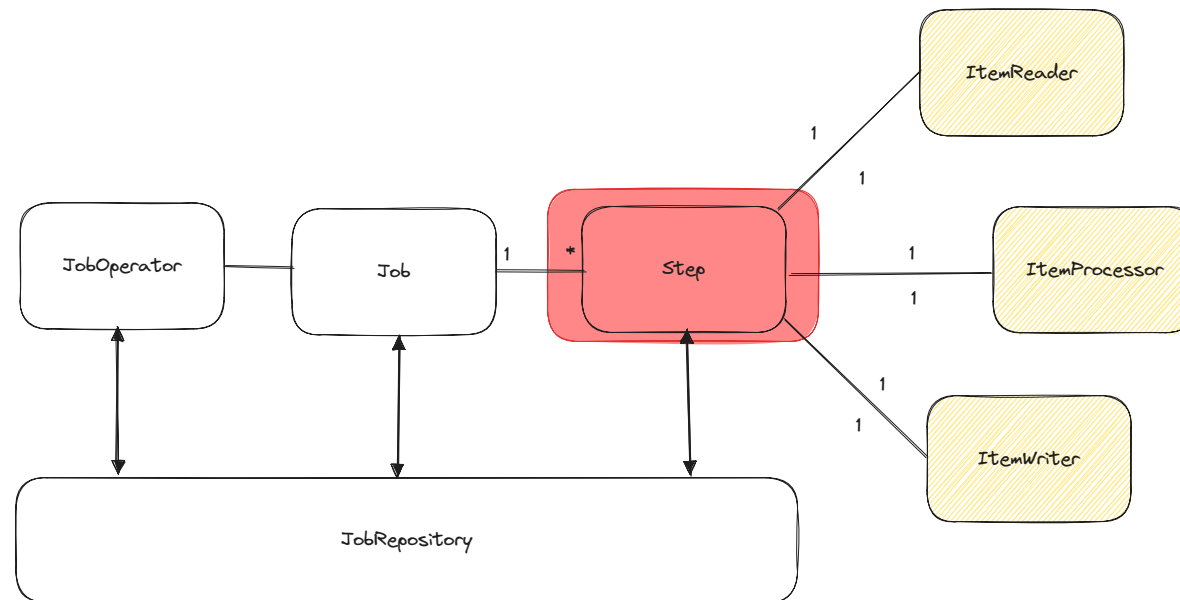


- Schnittstelle zur Verwaltung von Jobs
- Zur Laufzeit die erste Anlaufstelle zur Verwendung der Batch API

JOB OPERATOR (CODEBEISPIEL)

```
try {  
    // Retrieve JobOperator  
    JobOperator jobOperator = BatchRuntime.getJobOperator();  
  
    // Start job  
    long executionId = jobOperator.start("batchProcessingJob", null);  
  
    // Retrieve JobExecution and check status  
    JobExecution jobExecution = jobOperator.getJobExecution(executionId);  
    System.out.println("Job Status: " + jobExecution.getBatchStatus());  
  
    //...  
} catch (JobStartException e) {  
    e.printStackTrace();  
}
```

WAS IST EIN STEP?



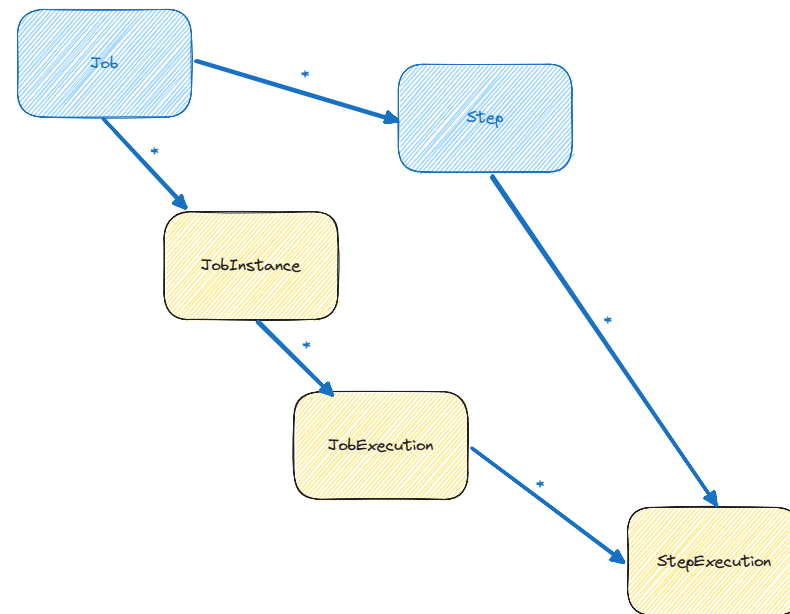
- Einzelne Verarbeitungsschritte eines **Jobs**
- Können task- oder datenorientiert sein

Step DEFINITION

- Erfolgt innerhalb der XML Job Definition
- Kann Verweis auf den nächsten **Step** enthalten

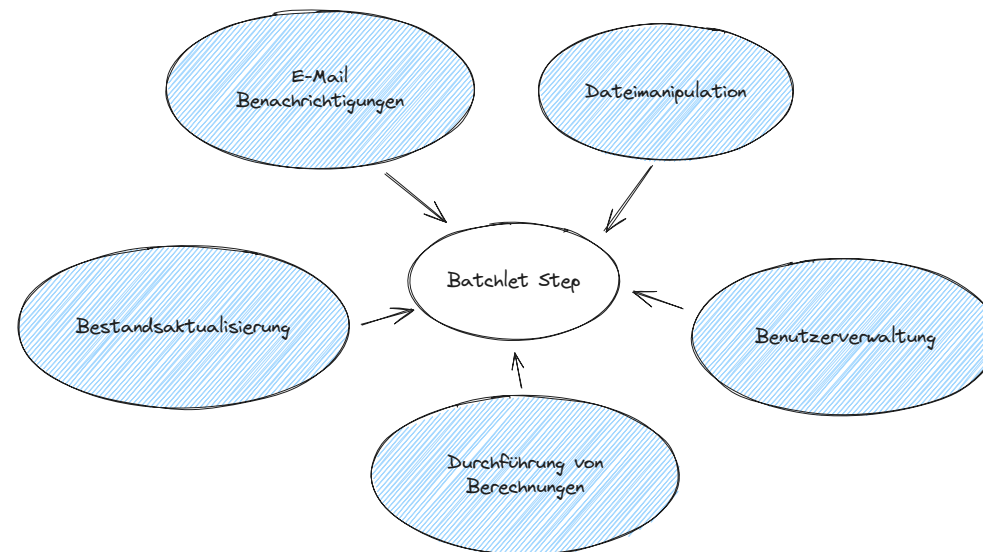
```
<job id="jobId">
  <step id="stepId" next="nextStepId">
    <!-- Stepdefinition -->
  </step>
  <step id="nextStepId">
    <!-- Stepdefinition -->
  </step>
</job>
```

STEPS ZUR LAUFZEIT



- repräsentiert eine einzelne Arbeitseinheit in einem Batch Job
- ein Step besteht aus:
 - Step Execution: ein einzelner Versuch einen Step auszuführen

TASK ORIENTIERTE STEPS: BACHTLETS



- Wird genutzt, um einfache, kurzlebige und nicht-transaktionale Aufgaben und Operationen durchzuführen.
- Bildet eine eigenständige Verarbeitungseinheit

INTERFACE Batchlet

```
package jakarta.batch.api;  
  
public interface Batchlet {  
    public String process() throws Exception;  
    public void stop() throws Exception;  
}
```

- Die Basisimplementierung **AbstractBatchlet** vereinfacht die Verwendung des Interfaces

CODEBEISPIEL: Batchlet

```
public class HelloBatchlet extends AbstractBatchlet {  
  
    @Override  
    public String process() {  
        System.out.println("Batchlet says Hello!");  
        return "OK";  
    }  
}
```

CODEBEISPIEL: JOBDEFINITION

```
<?xml version="1.0" encoding="UTF-8"?>
<job id="helloJob">
  <step id="helloStep">
    <batchlet ref="de.mathema.batch.hello.batchlet.HelloBatchlet" ></batchlet>
  </step>
</job>
```

BEISPIEL: HelloBatchlet

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("helloJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

BATCHLET AUFGABE

- Laden von Kundendaten aus einer CSV Datei in eine Datenbanktabelle
- Komplettverarbeitung durch ein Batchlet

INFOS ZUR DATENBANK

- h2 Datenbank starten:
 - .m2 repository finden
 - in den Ordner "repository" navigieren
 - ins Verzeichnis /com/h2database/h2/1.4.200 gehen
 - java -jar h2-1.4.200.jar ausführen
- DB Zugriff
 - DB Client: <http://localhost:8080>
 - DB URL: jdbc:h2:~/databases/customers
 - DB User: sa
 - Kein DB Passwort gesetzt

CHEATSHEET: DATENBANKZUGRIFF

```
// Öffnen einer Datenbankverbindung (am besten in try with resource, sonst close() nicht vergessen!)
Connection con = DriverManager.getConnection(config.getJdbcUrl(),config.getUsername(),
                                           config.getPassword());

// Erstellen einer Tabelle
Statement statement = con.createStatement();
statement.execute("create table if not exists tableName (" +
                 "someId varchar(255) PRIMARY KEY," +
                 "someValue varchar(255), " +
                 "someNumber integer)");

// Inhalt löschen (falls notwendig)
statement.execute("delete from tableName");

// Daten schreiben (alternativ auch im Batch möglich – siehe JavaDoc zu java.sql.PreparedStatement)
PreparedStatement pStmt = conn.prepareStatement("INSERT INTO customers (" +
                                                "someId, someValue, someNumber) VALUES (?, ?, ?) ");

pStmt.setString(1, "x235");
pStmt.setString(2, "aValue");
pStmt.setInt(3, 42);
pStmt.executeUpdate();

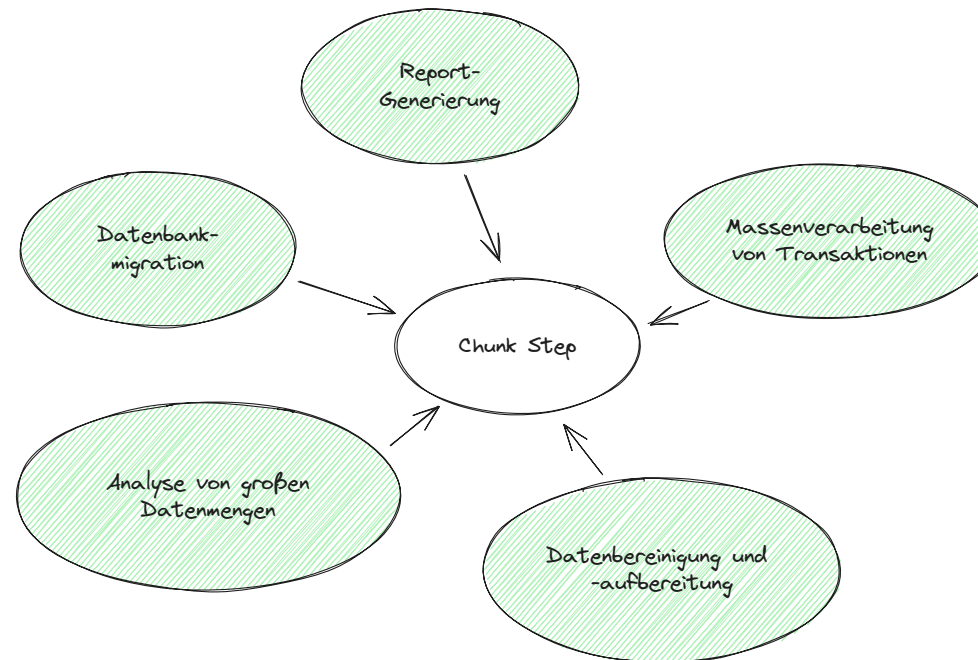
// Falls kein Auto-commit
con.commit();
```

CHEATSHEET: LESEN AUS DATEIEN

```
// Öffnen einer Datei zum Lesen (inkl. Reader)
BufferedReader reader = new BufferedReader(new InputStreamReader(
    this.getClass().getResourceAsStream("/file.csv"))
);

// Aus CSV lesen bis nichts mehr da ist...
String line;
while ( (line = reader.readLine()) != null ) {
    String[] data = line.split(",");
    // ...
}
```

DATENORIENTIERTE STEPS: CHUNKS



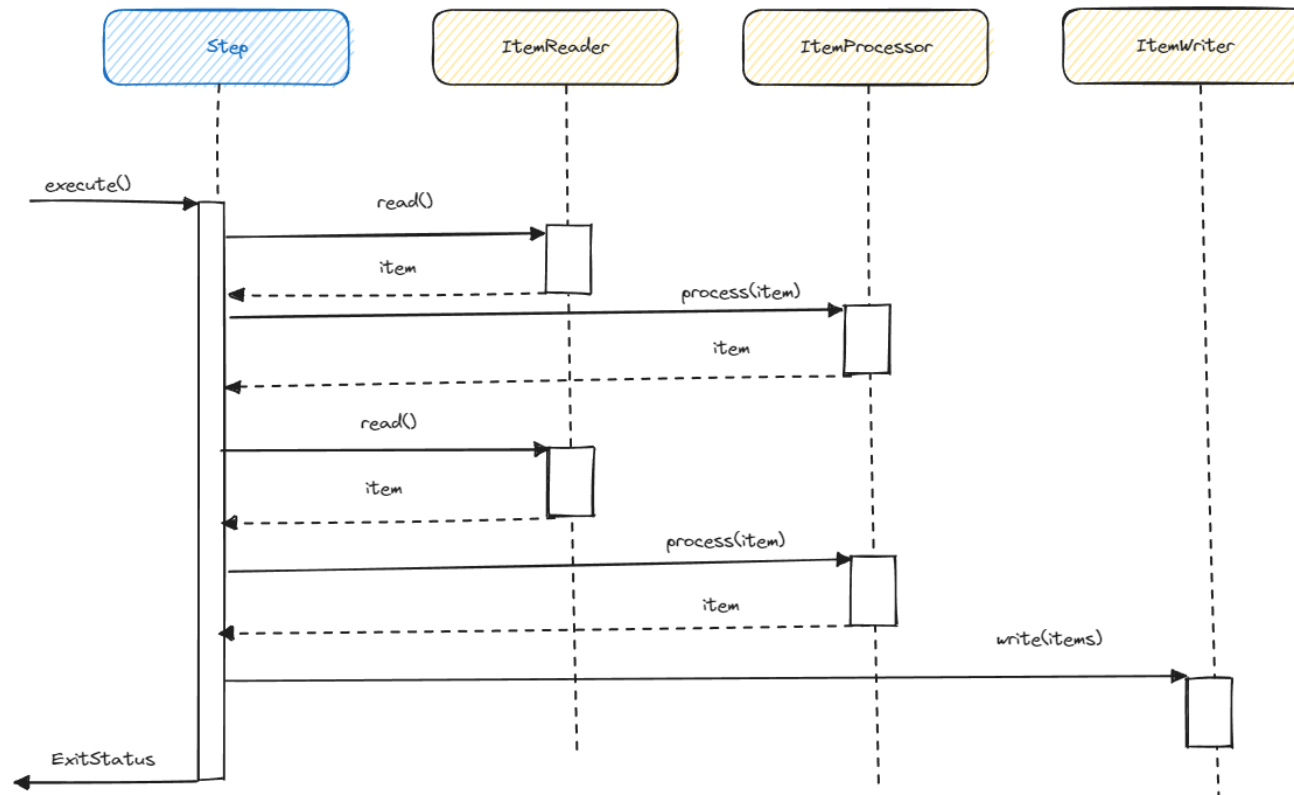
- Erste Wahl, um große Datenmengen transaktional zu lesen, zu verarbeiten und in einen Datenspeicher zu schreiben ("ETL")

CHUNK STEPS IM DETAIL

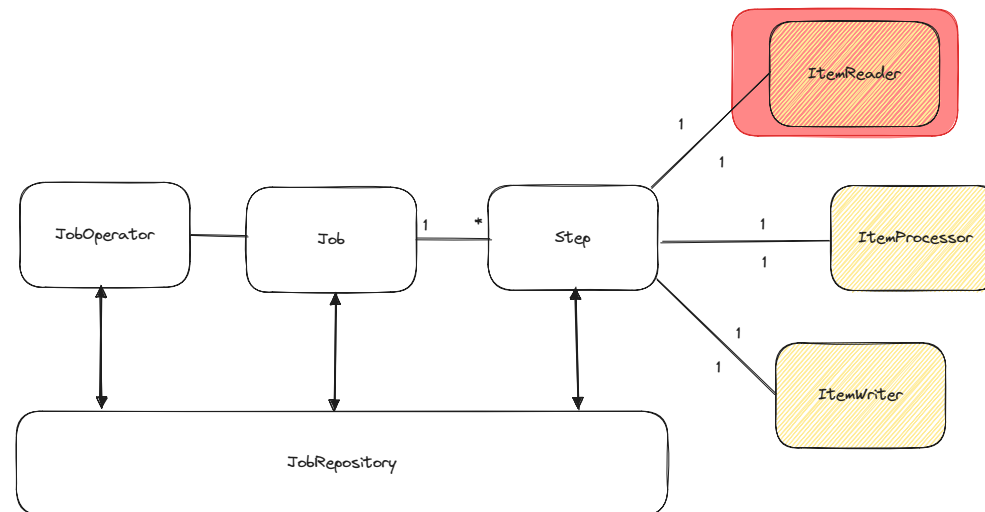
```
<job id="jobId">
  <step id="stepId">
    <chunk>
      <reader ref="..." ></reader>
      <processor ref="..." ></processor>
      <writer ref="..." ></writer>
    </chunk>
  </step>
</job>
```

- Verarbeitung zerfällt in drei Schritte
 - Daten laden (via `ItemReader`)
 - Daten verarbeiten (via `ItemProcessor`)
 - Daten schreiben (via `ItemWriter`)

CHUNK-WORKFLOW



Item Reader



- ist dafür verantwortlich Daten aus Datenquellen wie z.B. einer Datei, Datenbank oder einer Message Queue zu lesen

INTERFACE: Item Reader

```
package jakarta.batch.api.chunk;

public interface ItemReader {
    public void open(Serializable checkpoint) throws Exception;
    public void close() throws Exception;
    public Object readItem() throws Exception;
    public Serializable checkpointInfo() throws Exception;
}
```

- Die Basisimplementierung **AbstractItemReader** vereinfacht die Verwendung des Interfaces

CODEBEISPIEL: ItemReader

```
public class HelloReader extends AbstractItemReader {
    private final static List<String> NAMES = List.of("Ella", "Frank", "Patricia", "Victor", "World");
    private Iterator<String> nameIterator;

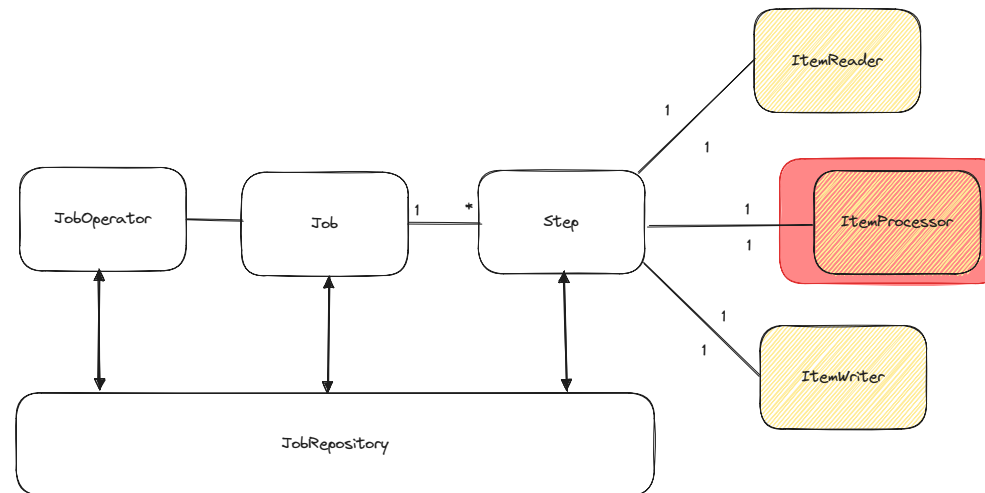
    @Override
    public void open(Serializable checkpoint) {
        nameIterator = NAMES.iterator();
    }

    @Override
    public Object readItem() {
        if(nameIterator != null && nameIterator.hasNext()) {
            return nameIterator.next();
        } else {
            return null;
        }
    }
}
```

CODEBEISPIEL: JOBDEFINITION

```
<job id="jobId">
  <step id="stepId">
    <chunk>
      <reader ref="HelloReader" ></reader>
      <processor ref="..." ></processor>
      <writer ref="..." ></writer>
    </chunk>
  </step>
</job>
```

Item Processor



- ist eine Komponente, die die Daten des Item Readers Element für Element verarbeitet und an den Item Writer übergibt

INTERFACE: ItemProcessor

```
package jakarta.batch.api.chunk;  
  
public interface ItemProcessor {  
    public Object processItem(Object item) throws Exception;  
}
```

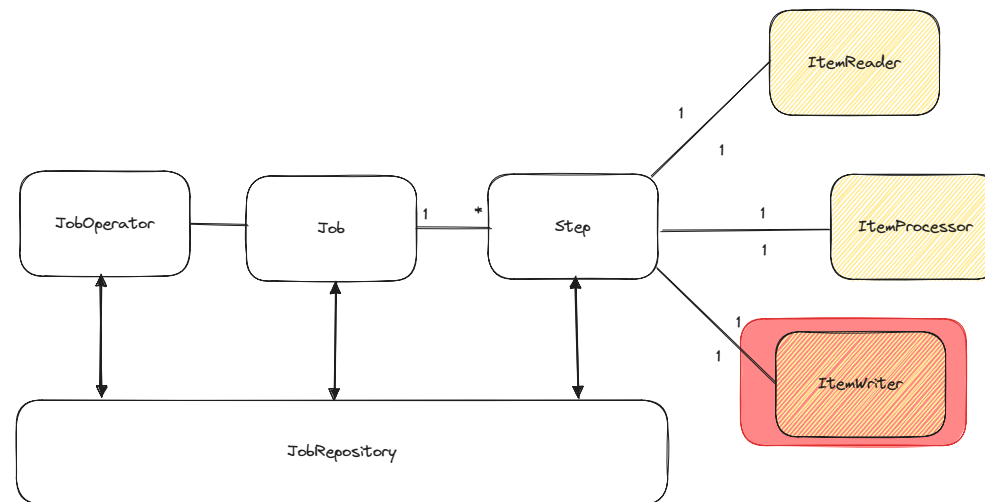
CODEBEISPIEL: ItemProcessor

```
public class HelloProcessor implements ItemProcessor {  
  
    @Override  
    public Object processItem(Object item) {  
        System.out.println("Processing " + item + "...");  
        return String.format("Hello %s!", item);  
    }  
}
```

CODEBEISPIEL: JOBDEFINITION

```
<job id="jobId">
  <step id="stepId">
    <chunk>
      <reader ref="HelloReader" ></reader>
      <processor ref="HelloProcessor" ></processor>
      <writer ref="..." ></writer>
    </chunk>
  </step>
</job>
```

Item Writer



- ist verantwortlich dafür die verarbeiteten Daten des Item Processors in einen Datenspeicher zu schreiben wie z.B. Datenbank, Datei oder Message Queue

INTERFACE: Item Writer

```
package jakarta.batch.api.chunk;

public interface ItemWriter {
    public void open(Serializable checkpoint) throws Exception;
    public void close() throws Exception;
    public void writeItems(List<Object> items) throws Exception;
    public Serializable checkpointInfo() throws Exception;
}
```

- Die Basisimplementierung **AbstractItemWriter** vereinfacht die Verwendung des Interfaces

CODEBEISPIEL: ItemWriter

```
public class HelloWriter extends AbstractItemWriter {  
  
    @Override  
    public void writeItems(List<Object> items) {  
        items.forEach(System.out::println);  
    }  
}
```

CODEBEISPIEL: JOBDEFINITION

```
<job id="jobId">
  <step id="stepId">
    <chunk>
      <reader ref="HelloReader" ></reader>
      <processor ref="HelloProcessor" ></processor>
      <writer ref="HelloWriter" ></writer>
    </chunk>
  </step>
</job>
```

BEISPIEL: HelloChunk

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("helloJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

CHUNK AUFGABE

- Laden von Kundendaten aus einer CSV Datei in eine Datenbanktabelle
- Aufteilung der Verarbeitungsschritte in
 - Laden
 - Verarbeiten
 - Schreiben

CHUNK STEPS: CHECKPOINTS

- **Item Reader** und **Item Writer** können Ihren Fortschritt mittels **Checkpoints** festhalten.
- Im Falle eines Laufzeitfehlers kann am letzten erfolgreichen **Checkpoint** wiederaufgesetzt werden.
- Ermöglicht Retries und das Wiederanlaufen eines mit Fehler beendeten Jobs.

CHUNK STEPS: CHECKPOINTS DEFINIEREN

```
public class HelloReader extends AbstractItemReader {  
  
    public static class HelloCheckpoint implements Serializable {  
        private int lastItemProcessed = -1;  
    }  
  
    // ...  
}
```

- Ein Checkpoint enthält Information über den vom **ItemReader** erfolgreich erreichten Zustand.
- Die konkrete Ausprägung ist abhängig von der fachlichen Logik des **Readers**.

CHUNK STEPS: CHECKPOINTS SETZEN UND AUSLESEN

```
public class HelloReader extends AbstractItemReader { ...

    @Override
    public void open(Serializable checkpoint) {
        helloCheckpoint = checkpoint != null ? (HelloCheckpoint) checkpoint : new HelloCheckpoint();
        if((helloCheckpoint).lastItemProcessed > -1) { ... }
    }

    @Override
    public Object readItem() {
        ...
        helloCheckpoint.lastItemProcessed++;
        ...
    }

    @Override
    public Serializable checkpointInfo() { return helloCheckpoint; }
}
```

- Checkpoints werden von der Batch Runtime abgefragt und ggf. wieder bereitgestellt

CHECKPOINT ALGORITHMUS

```
public class SimpleCheckpointAlgorithm implements CheckpointAlgorithm {  
  
    private int numOfRounds = 0;  
  
    @Override  
    public int checkpointTimeout() throws Exception { return 0; }  
  
    @Override  
    public void beginCheckpoint() {}  
  
    @Override  
    public boolean isReadyToCheckpoint() throws Exception {  
        numOfRounds++;  
        return numOfRounds%2==0;  
    }  
  
    @Override  
    public void endCheckpoint() throws Exception {}  
}
```

- Ermittlung der Zeitpunkte zur Erstellung eines neuen Checkpoints.

SETZEN DES CHECKPOINT ALGORITHMUS

```
<job id="helloJob" ... >
  <step id="helloStep" ... >
    <chunk checkpoint-policy="custom" >
      ...
      <checkpoint-algorithm ref="..." ></checkpoint>
    </chunk>
  </step>
</job>
```

- Im Rahmen der **Chunk Step** Definition kann u.a. auch der zu verwendende Checkpoint Algorithmus definiert werden.

CHUNK STEPS: FEHLERARTEN SPEZIFIZIEREN

```
<job id="helloJob" ... >
  <step id="helloStep" ... >
    <chunk ... >
      ...
      <skipable-exception-classes>
        <include class="java.lang.UnsupportedOperationException"></include>
      </skipable-exception-classes>
      <retryable-exception-classes>
        <include class="java.lang.IllegalArgumentException"></include>
      </retryable-exception-classes>
    </chunk>
  </step>
</job>
```

- Exceptions können das Chunk Verhalten verändern.
- `skipable-exception-classes` definiert eine Liste von Exceptions, bei denen der Fehler übersprungen wird.
- `retryable-exception-classes` definiert eine Liste von Exceptions, bei denen ein automatischer `Retry` möglich ist.

CHUNK STEPS: SKIPS & RETRIES BEGRENZEN

```
<job id="helloJob" ... >
  <step id="helloStep" ... >
    <chunk skip-limit="10" retry-limit = "50" >
      ...
      <skipable-exception-classes>
        ...
      </skipable-exception-classes>
      <retryable-exception-classes>
        ...
      </retryable-exception-classes>
    </chunk>
  </step>
</job>
```

- Ein **Chunk Step** kann die zulässige Anzahl für **Skips** bzw. **Retries** begrenzen.
- Die maximale Anzahl kann in der Batch Job Definition angegeben werden.
- Werden keine Werte angegeben, so besteht auch kein Limit.

CHUNK STEPS: RETRY-VERHALTEN ANPASSEN

```
<job id="helloJob" ... >
  <step id="helloStep" ... >
    <chunk ... >
      ...
      <skipable-exception-classes>
        ...
      </skipable-exception-classes>
      <retryable-exception-classes>
        ...
      </retryable-exception-classes>
      <no-rollback-exception-classes>
        <include class="java.lang.IllegalArgumentException" ></include>
      </no-rollback-exception-classes>
    </chunk>
  </step>
</job>
```

- Standardmässig führt ein **Retry** zu einem **Rollback** des gesamten **Chunks**. D.h. Rücksprung zum letzten **Checkpoint**.
- Dieses Verhalten kann in der Job Definition geändert werden.

CODEBEISPIEL: JOBDEFINITION MIT SKIP & RETRY

```
<job id="helloJob" restartable="true">
  <step id="helloStep">
    <chunk checkpoint-policy="custom">
      <reader ref="de.mathema.batch.hello.chunk.HelloReader" ></reader>
      <processor ref="de.mathema.batch.hello.chunk.HelloProcessor" ></processor>
      <writer ref="de.mathema.batch.hello.chunk.HelloWriter" ></writer>
      <checkpoint-algorithm
        ref="de.mathema.batch.hello.chunk.SimpleCheckpointAlgorithm" ></checkpoint>
      <skipable-exception-classes>
        <include class="java.lang.UnsupportedOperationException"></include>
      </skipable-exception-classes>
      <retryable-exception-classes>
        <include class="java.lang.IllegalArgumentException"></include>
      </retryable-exception-classes>
    </chunk>
  </step>
</job>
```

BEISPIEL: RestartableHelloChunk

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("helloJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

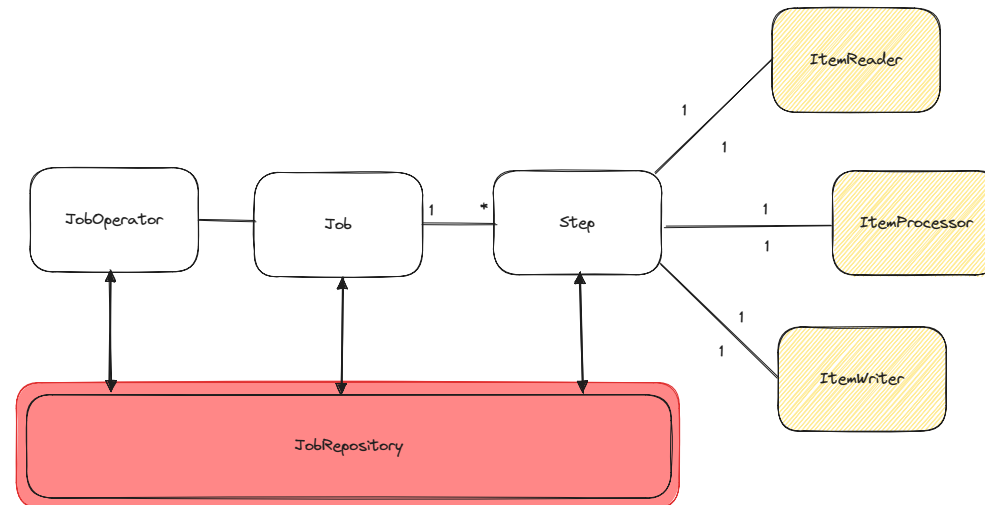
// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

CHUNK RETRY AUFGABE:

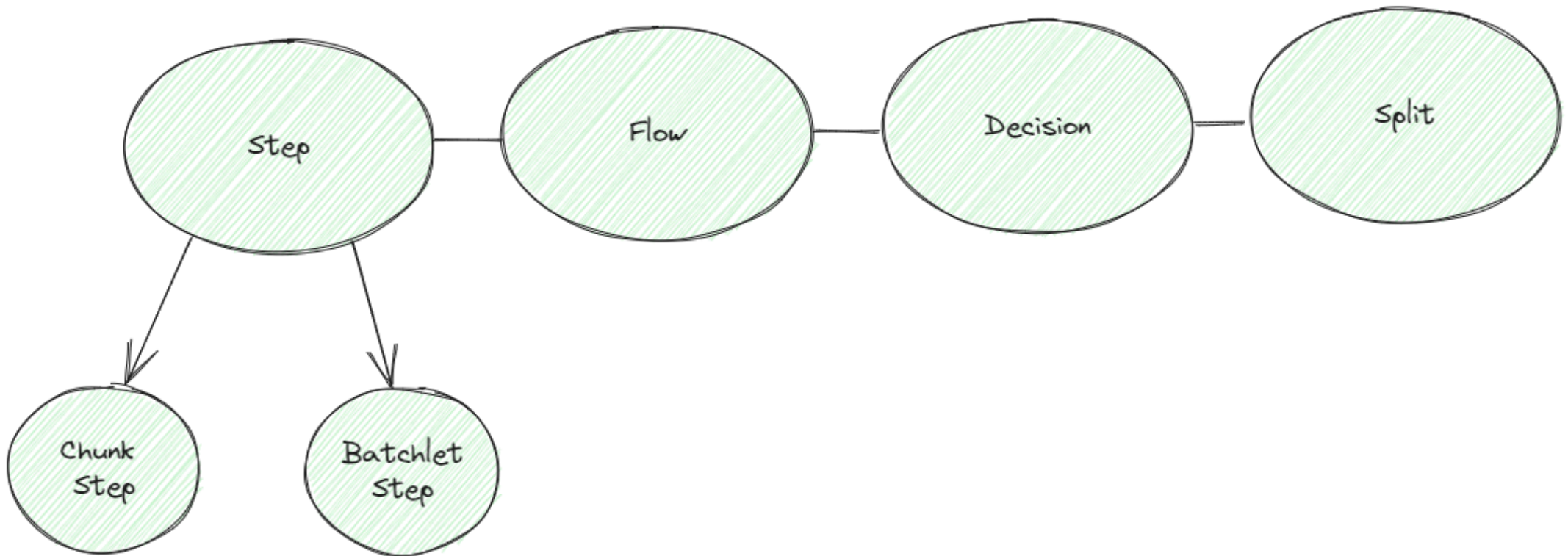
- Laden von Kundendaten aus einer CSV Datei in eine Datenbanktabelle
- Setzen eines Checkpoints alle 25 Datensätze
- Definition einer beliebigen **RuntimeException** als *skipable*
 - Provozieren der "Skip-Exception" bei der Verarbeitung von Kunden mit dem Vornamen "Michael"
- Definition einer beliebigen **RuntimeException** als *retryable*
 - Provozieren der "Retry-Exception" bei ca. jedem zwanzigstem Read
- Was passiert, wenn die "Retry-Exception" die Transaktion nicht zurückrollt?
- Was passiert wenn ein **Skip**- oder **Retry-Limit** gesetzt ist?

WAS IST EIN JOB REPOSITORY?



- beinhaltet die Informationen über die Jobs, die derzeit laufen oder gelaufen sind wie beispielsweise Job und Step Execution Historie oder Zustandsverfolgung

WEITERE STEP-ARTEN

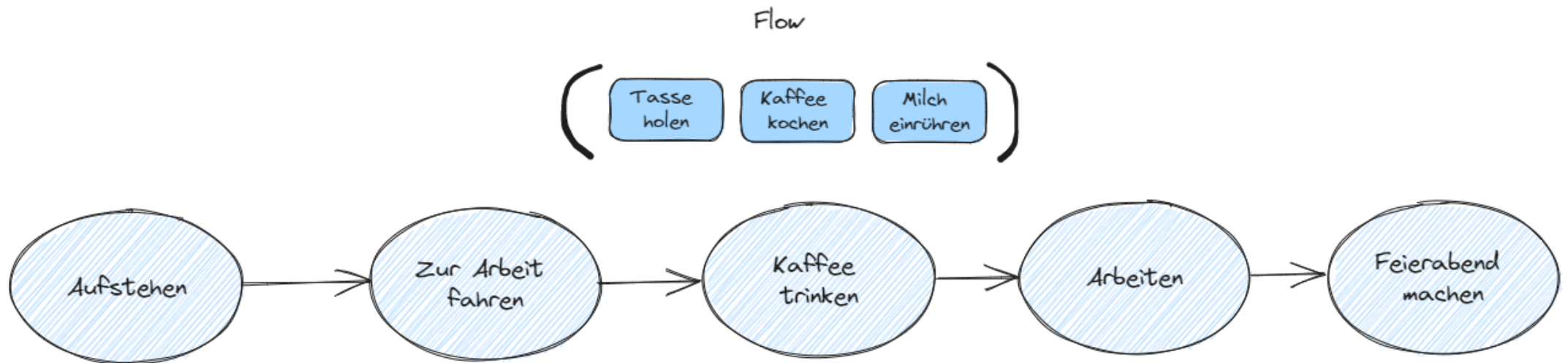


FLOW



- Definiert eine Folge von **Steps**, die zusammen als logische Einheit ausgeführt werden
- Die **Steps** eines **Flows** können nur innerhalb des **Flows** weiter navigieren
- Der **Flow** definiert wie es nach seiner Ausführung weiter geht (NICHT der letzte Step)

SUB-FLOWS



- Ein **Flow** kann beliebige **Steps** und damit auch weitere **Flows** enthalten

FLOW-DEFINITION

```
<job id="workdayJob" restartable="false">
  <flow id="Arbeitstag">
    <step id="Aufstehen" next="ZurArbeitFahren"> ... </step>
    <step id="ZurArbeitFahren" next="KaffeeTrinken"> ... </step>
    <flow id="KaffeeTrinken" next="Arbeiten">
      <step id="TasseHolen" next="KaffeeKochen"> ... </step>
      <step id="KaffeeKochen" next="MilchEinruehren"> ... </step>
      <step id="MilchEinruehren"> ... </step>
    </flow>
    <step id="Arbeiten" next="FeierabendMachen"> ... </step>
    <step id="FeierabendMachen"> ... </step>
  </flow>
</job>
```

BEISPIEL: WORKDAYFLOW

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("workdayJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

FLOW AUFGABE:

- Kunden haben zwei neue Attribute:
 - entryDate
 - revenue
- Flow #1
 - Kunden aus CSV lesen und in DB schreiben (Chunk)
- Flow #2
 - Neue Spalte einfügen mit dem Namen 'membership' (Batchlet)
- Flow #3
 - Mitgliedschaft berechnen und eintragen (Chunk)
 - im Processor soll ermittelt werden, ob der Kunde schon länger als 7 Jahre bei uns Kunde ist
 - falls ja ⇒ Membership Spalte 'Premium'
 - falls nein ⇒ Membership Spalte 'Basic'

CHEATSHEET: DATENBANKZUGRIFF

```
// Öffnen einer Datenbankverbindung (am besten in try with resource, sonst close() nicht vergessen!)
Connection con = DriverManager.getConnection(config.getJdbcUrl(),config.getUsername(),
                                           config.getPassword());

// Hinzufügen einer Spalte zu einer bestehenden Tabelle
Statement statement = con.createStatement();
statement.execute("ALTER TABLE customers ADD IF NOT EXISTS membership varchar(255));
```

DECISION STEP



- Bisher: statische Definition der **Step** Abfolge via **next**.
- **Decision** Elemente bieten die Möglichkeit den weiteren Ablauf dynamisch, auf Basis der Ergebnis der vorhergehenden **Step** Ausführung zu bestimmen.
- Entscheidung wird in einem **Decider** Artefakt implementiert.
- Rückgabewert des Artefakts wird in XML auf Verhalten gemapped.

INTERFACE: DECIDER

```
package jakarta.batch.api;  
  
public interface Decider {  
    public String decide(StepExecution[] executions) throws Exception;  
}
```

CODEBEISPIEL: DECIDER

```
public class RabattDecider implements Decider {  
  
    @Override  
    public String decide(StepExecution[] executions) {  
        return switch (executions[0].getExitStatus()) {  
            case "ACTIVE" -> "MEMBER";  
            case "CANCELED" -> "STANDARD";  
            case "REQUESTED" -> "MEMBER";  
            case "NONE" -> "STANDARD";  
            default -> "STANDARD";  
        };  
    }  
}
```

BEISPIEL: XML-KONFIGURATION

```
<job id="KaufpreisBerechnenJob">
  ...
  <step id="KaeufermitgliedschaftUeberpruefen" next="RabattDecision"> ... </step>

  <decision id="RabattDecision" ref="de.mathema.batch.decision.RabattDecider">
    <next on="MEMBER" to="PremiumRabattBerechnen"></next>
    <next on="STANDARD" to="StandardRabattBerechnen"></next>
  </decision>

  <step id="PremiumRabattBerechnen"> ... </step>

  <step id="StandardRabattBerechnen"> ... </step>
</job>
```

BEISPIEL: DECISION

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("priceCalcJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

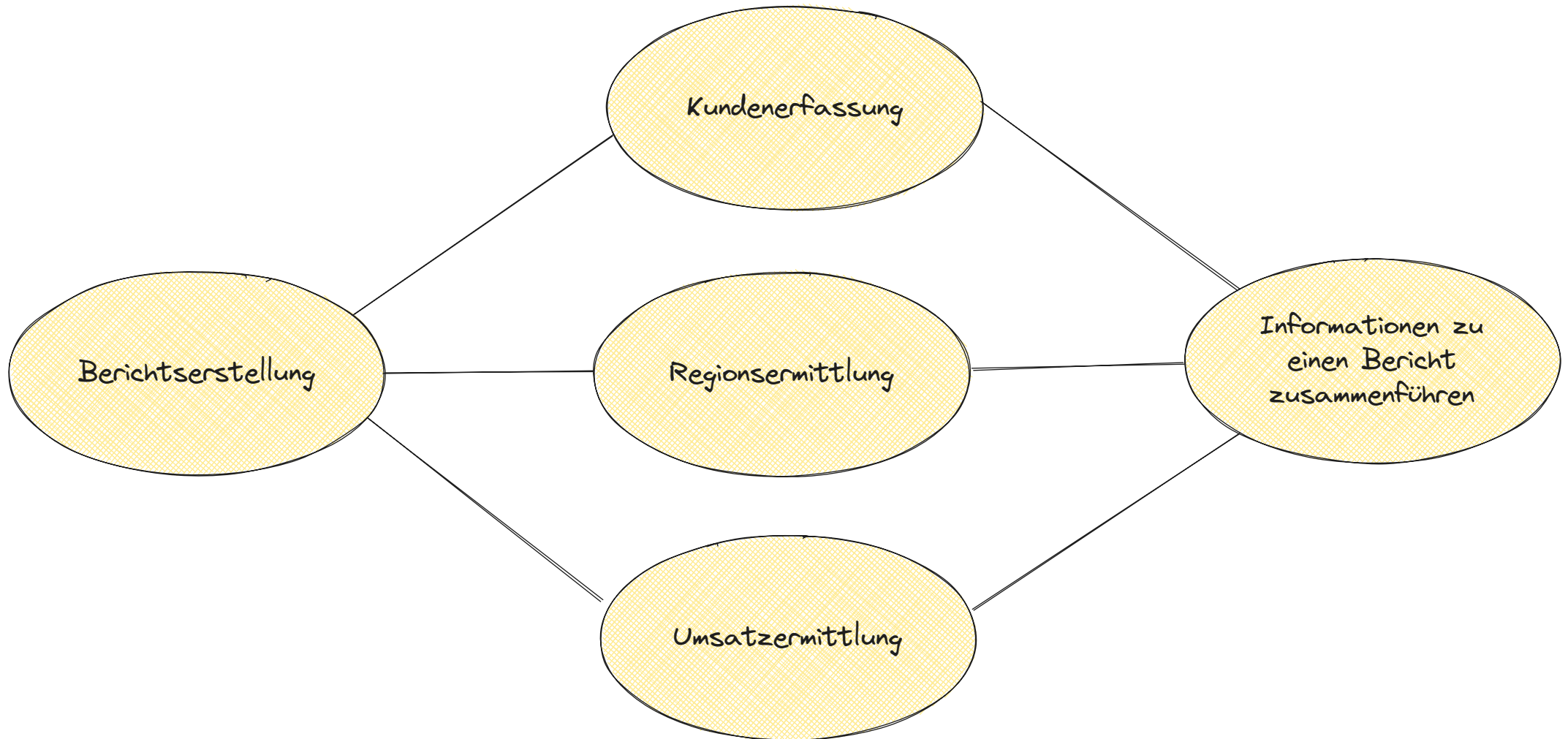
// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

DECISION AUFGABE:

- Erweiterung der letzten Aufgabe
- Neben der **Membership** Spalte wird via weiterem **Batchlet** eine weitere Spalte **Discount** eingeführt (Erweiterung `addColumnFlow`)
- Nach der Ausführung des `MembershipFlows` wird ein neuer `DiscountFlow` definiert, der für jeden Kunden im Chunk folgenden Discount in der DB speichert
 - `Membership == Basic` $\Rightarrow$ 10%
 - `Membership == Premium` $\Rightarrow$ 20%
- Einführen einer Decision
 - Wenn Kunde in München wohnt, dann extra `DiscountFlow`
- Im `ExtraDiscountFlow` bekommt jeder Kunde aus München einen ZUSÄTZLICHEN Extra Discount von 20%

SPLIT



- Definiert eine Gruppe **unterschiedlicher** Flows, die gleichzeitig ausgeführt werden

BEISPIEL: XML-KONFIGURATION

```
<job id="splittedJob">
  <split id="parallelWork">
    <flow id="flow1">
      <step id="work1">
        <batchlet ref="de.mathema.batch.split.batchlet.SlowWorkingBatchlet" ></batchlet>
      </step>
    </flow>

    <flow id="flow2">
      <step id="work2">
        <batchlet ref="de.mathema.batch.split.batchlet.FastWorkingBatchlet" ></batchlet>
      </step>
    </flow>

    <flow id="flow3">
      <step id="work3">
        <batchlet ref="de.mathema.batch.split.batchlet.MediumWorkingBatchlet"></batchlet>
      </step>
    </flow>
  </split>
</job>
```

BEISPIEL: SPLIT

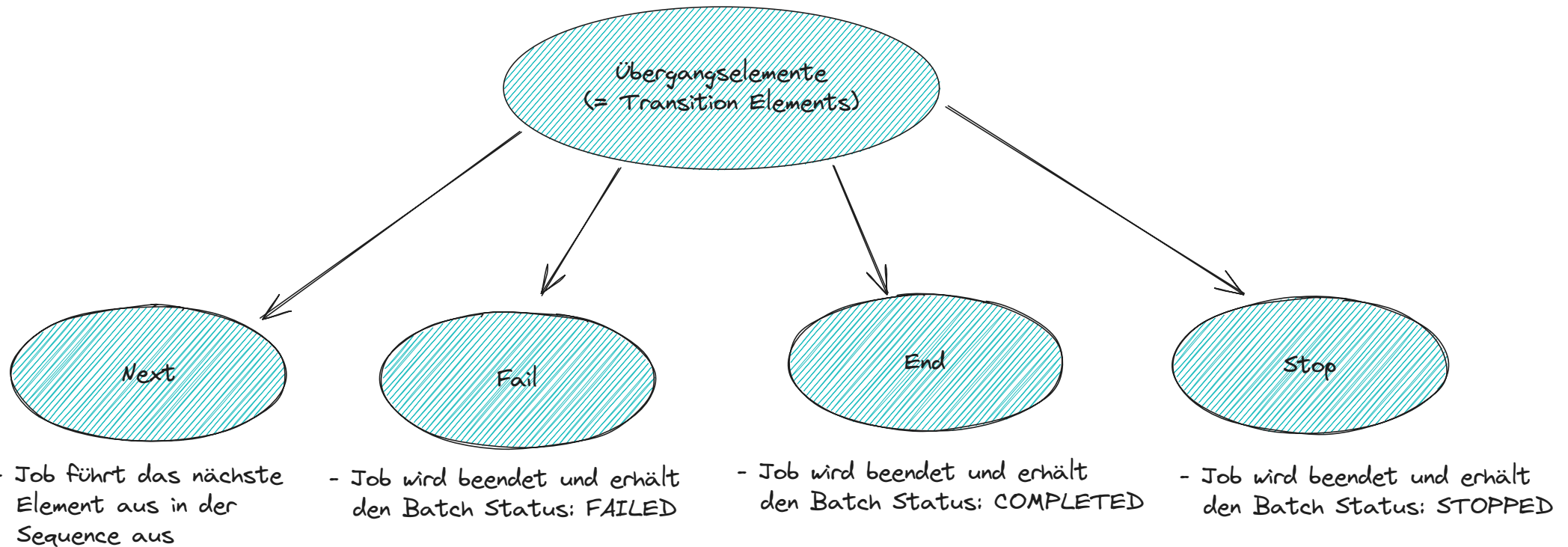
```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("splittedJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

ÜBERGANGSELEMENTE



- Basierend auf dem **ExitStatus** eines **Steps** kann der weitere Ablauf des gesamten **Jobs** beeinflusst werden.

FLOW-DEFINITION MIT TRANSITIONEN

```
<job id="workdayJob" restartable="false">
  <flow id="Arbeitstag">
    <step id="Aufstehen" next="ZurArbeitFahren">
      ...
    <end on="SUN" exit-status="Heute ist Sonntag! :-)"></end>
  </step>
  <step id="ZurArbeitFahren" next="KaffeeTrinken"> ... </step>
  <flow id="KaffeeTrinken" next="Arbeiten">
    <step id="TasseHolen" next="KaffeeKochen"> ... </step>
    <step id="KaffeeKochen" next="MilchEinruehren">
      ...
    <fail on="DECAF" exit-status="Nur entkoffeinierter Kaffee da! :-("></fail>
  </step>
  <step id="MilchEinruehren"> ... </step>
</flow>
<step id="Arbeiten" next="FeierabendMachen"> ... </step>
<step id="FeierabendMachen"> ... </step>
</job>
```

BEISPIEL: WORKDAYFLOW MIT AUSSTIEG

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("workdayJob", null);

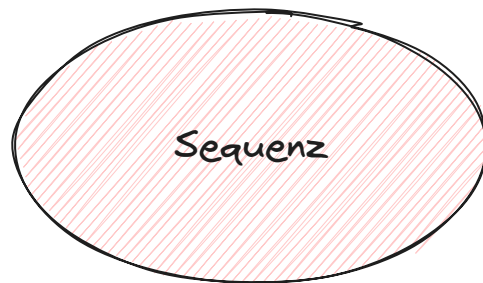
JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

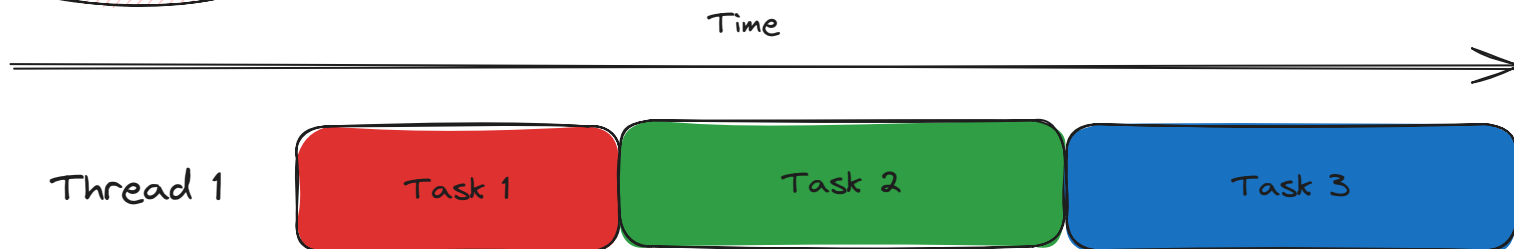
System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

JOB VERARBEITUNG

SEQUENZ



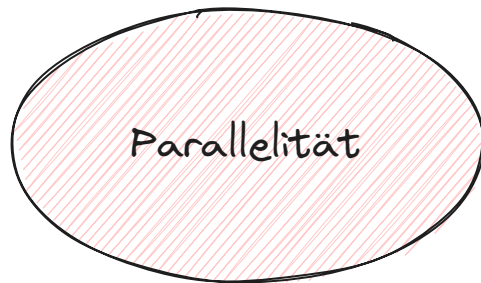
- mehrere Aufgaben werden nacheinander Schritt für Schritt bearbeitet



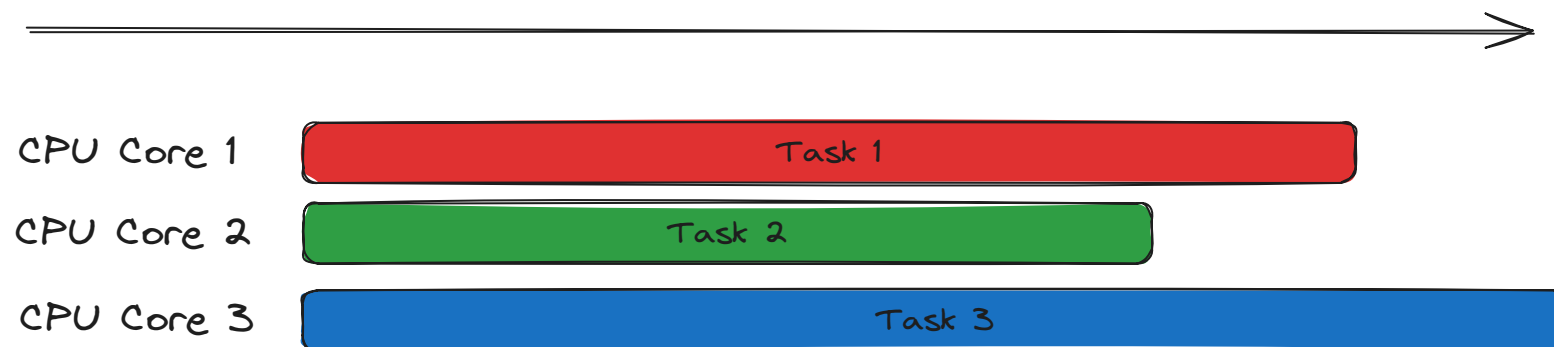
SEQUENZ - VOR UND NACHTEILE

- Fast alles bisherigen Beispiele (außer: `split`) liefen sequentiell ab
- PRO: Einfaches Programmiermodell, keine "Nebenläufigkeitseffekte"
- CONTRA: Langsame Verarbeitung, Hardware Kapazitäten werden nicht ausgenutzt

PARALLELITÄT



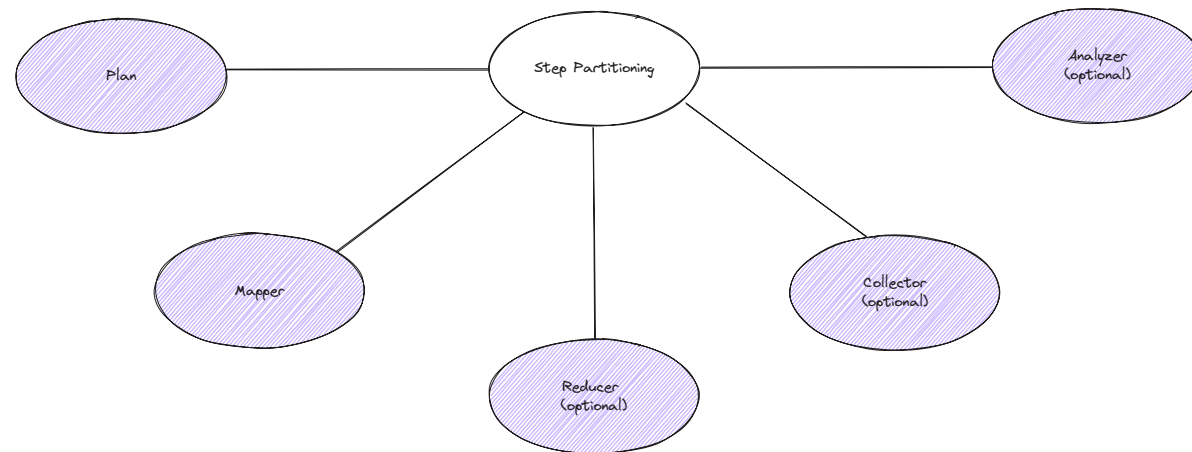
- mehrere Aufgaben werden zur selben Zeit bearbeitet



PARALLELITÄT - VOR- UND NACHTEILE

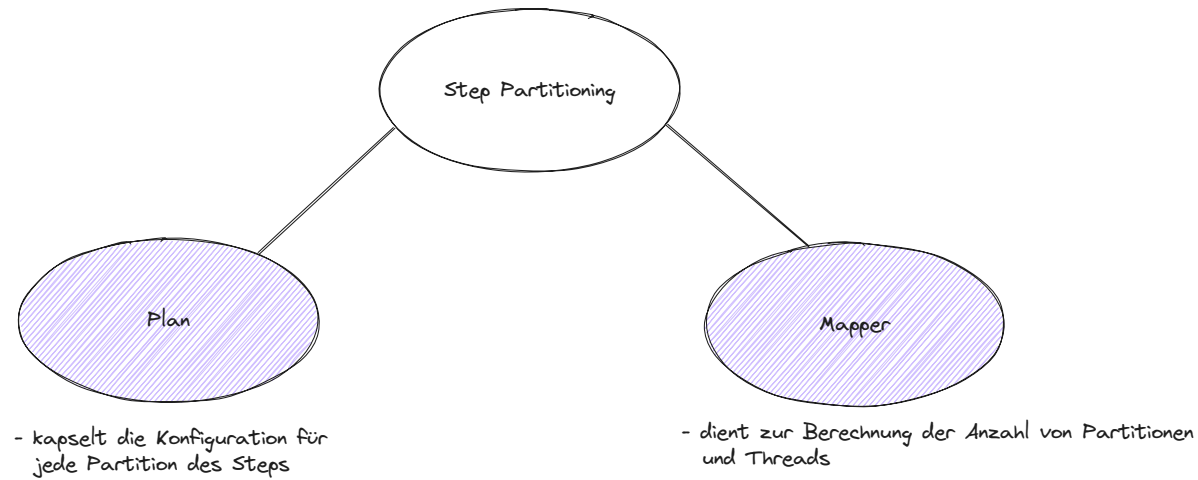
- Nur im `split`Beispiel liefen bisher unabhängige **Flows** parallel
- PRO: Schnellere Verarbeitung, Ausnutzung der Hardwarekapazitäten
- CONTRA: Aufteilung notwendig, Nebenläufigkeitseffekte, Zusammenführen der Ergebnisse u.U. aufwendig

STEP PARTITIONING



- Technik, um eine große Datenmenge in kleinere Teile (=Partitionen) aufzuspalten und sie mithilfe mehrerer Threads parallel zu verarbeiten
- Sorgt für eine schnellere Bearbeitungszeit

STEP PARTITIONING (NOTWENDIGE ANTEILE)



PARTITION PLAN (STATISCH)

```
<step id="stepId">
  <chunk>
    <reader ref="...">
      <properties>
        <property name="firstItem" value="#{partitionPlan['firstItem']}"></property>
        <property name="lastItem" value="#{partitionPlan['lastItem']}"></property>
      </properties>
    </reader>
    <processor ref="..." ></processor>
    <writer ref="..." ></writer>
  </chunk>
  <partition>
    <plan partitions="2">
      <properties partition="0">
        <property name="firstItem" value="0"></property>
        <property name="lastItem" value="99"></property>
      </properties>
      <properties partition="1">
        <property name="firstItem" value="100"></property>
        <property name="lastItem" value="200"></property>
      </properties>
    </plan>
  </partition>
</step>
```

PARTITION MAPPER (DYNAMISCH) - XML

```
<step id="stepId">
  <chunk>
    <reader ref="...">
      <properties>
        <property name="firstItem" value="#{partitionPlan['firstItem']}"></property>
        <property name="lastItem" value="#{partitionPlan['lastItem']}"></property>
      </properties>
    </reader>
    <processor ref="..." ></processor>
    <writer ref="..." ></writer>
  </chunk>
  <partition>
    <mapper ref="...">
      <properties>
        <property name="strategy" value="STANDARD"></property>
      </properties>
    </mapper>
  </partition>
</step>
```

PARTITION MAPPER (DYNAMISCH) - INTERFACE

```
package jakarta.batch.api.partition;  
  
import jakarta.batch.api.partition.PartitionPlan;  
  
public interface PartitionMapper {  
    public PartitionPlan mapPartitions() throws Exception;  
}
```

CODEBEISPIEL: PARTITIONED READER

```
public class HelloPartitionedReader extends AbstractItemReader {
    @Inject
    @BatchProperty
    private int firstItem;

    @Inject
    @BatchProperty
    private int lastItem;

    private Iterator<String> nameIterator;

    @Override
    public void open(Serializable checkpoint) {
        nameIterator = HelloData.NAMES.stream().
            skip(firstItem).limit(lastItem-firstItem+1).iterator();
    }

    @Override
    public Object readItem() throws InterruptedException { /* ... */ }
}
```

CODEBEISPIEL: PARTITIONMAPPER

```
public class HelloPartitionMapper implements PartitionMapper {

    @Override
    public PartitionPlan mapPartitions() {
        int numOfItems = HelloData.NAMES.size();
        int numOfPartitions;

        // Computing numOfPartitions...

        PartitionPlan plan = new PartitionPlanImpl();
        plan.setPartitions(numOfPartitions);
        Properties[] partProps = new Properties[numOfPartitions];

        for(int i=0;i<numOfPartitions;i++){
            Properties props = new Properties();
            props.put("partitionNumber", i);
            // Define more Properties...
        }
        plan.setPartitionProperties(partProps);
        return plan;
    }
}
```

BEISPIEL: PARTITIONED HELLO CHUNK

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("helloJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

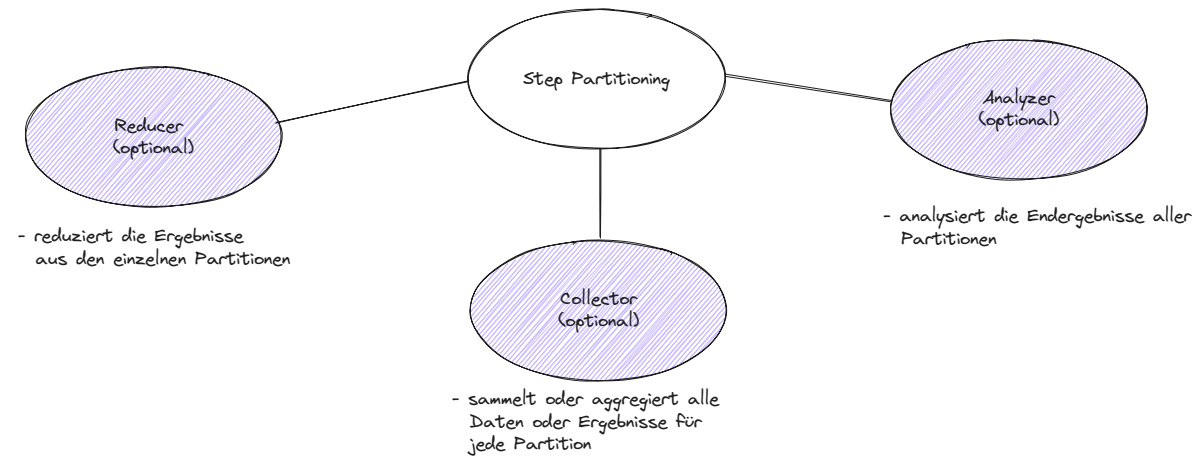
// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

PARTITIONED CHUNK AUFGABE (TEIL 1)

- Laden von Kundendaten aus einer CSV Datei in eine Datenbanktabelle (analog Chunk Beispiel)
- Teile die Verarbeitung in fünf Partitionen auf
 - per XML
 - per Partition Mapper

STEP PARTITIONING (OPTIONALE ANTEILE)



PartitionCollector

```
package jakarta.batch.api.partition;  
import java.io.Serializable;  
  
public interface PartitionCollector {  
    public Serializable collectPartitionData() throws Exception;  
}
```

- Datenquelle um Ergebnisse aus einer Partition an den "Hauptjob" zurückzugeben
- Sammelt die Ergebnisse einer Partition und übergibt sie an den **Ana**lyzer (auf dem "Main Thread")

CODEBEISPIEL: HelloPartitionCollector

```
public class HelloPartitionCollector implements PartitionCollector {  
  
    @Inject  
    private StepContext sCtx;  
  
    @Override  
    public Serializable collectPartitionData() {  
        System.out.println("Collecting...\n" + sCtx.getPersistentUserData());  
        return sCtx.getPersistentUserData();  
    }  
}
```

PartitionAnalyzer

```
package jakarta.batch.api.partition;

import java.io.Serializable;
import jakarta.batch.runtime.BatchStatus;

public interface PartitionAnalyzer {

    public void analyzeCollectorData(Serializable data) throws Exception;

    public void analyzeStatus(BatchStatus batchStatus, String exitStatus) throws Exception;
}
```

- Verarbeitet die Ergebnisse (Daten und/oder Endzustände) aller Partitionen
- Datensenke, um die Daten der Partitionen dem Hauptjob zugänglich zu machen

CODEBEISPIEL: HelloPartitionAnalyzer

```
public class HelloPartitionAnalyzer extends AbstractPartitionAnalyzer {  
  
    @Inject  
    private StepContext sCtx;  
  
    @Override  
    public void analyzeCollectorData(Serializable data) throws Exception {  
        System.out.println("Analyzing...\n" + data);  
  
        if(sCtx.getPersistentUserData() == null){  
            sCtx.setPersistentUserData(new ArrayList<String>());  
        }  
        ((List<String>)sCtx.getPersistentUserData()).addAll((List<String>)data);  
    }  
}
```

PartitionReducer

```
package jakarta.batch.api.partition;

public interface PartitionReducer {

    public enum PartitionStatus {COMMIT, ROLLBACK}

    public void beginPartitionedStep() throws Exception;
    public void beforePartitionedStepCompletion() throws Exception;
    public void rollbackPartitionedStep() throws Exception;
    public void afterPartitionedStepCompletion(PartitionStatus status) throws Exception;

}
```

- Bildet eine Klammer um den partitionierten **Step**.
- Kann vor der Partitionierung die Umgebung vorbereiten.
- Kann das Ergebnis der Partitionierung verändern bzw. den nächsten Schritten zugänglich machen.

HelloPartitionReducer

```
public class HelloPartitionReducer extends AbstractPartitionReducer {

    @Inject
    private StepContext sCtx;

    @Inject
    private JobContext jCtx;

    @Override
    public void afterPartitionedStepCompletion(PartitionStatus status) {
        System.out.println("Step completed: " + status);
    }

    @Override
    public void beforePartitionedStepCompletion() {
        System.out.println("Making result globally available...");
        jCtx.setTransientUserData(sCtx.getPersistentUserData());
    }

    @Override
    public void beginPartitionedStep() {
        System.out.println("Let's start the partition...");
    }
}
```

CODEBEISPIEL: DATENVERARBEITUNG IN FOLGESCHRITT

```
public class HelloBatchlet extends AbstractBatchlet {  
  
    @Inject  
    private JobContext jCtx;  
  
    @Override  
    public String process() throws Exception {  
        if(jCtx.getTransientUserData() == null ) {  
            System.out.println("No data found! :-(");  
            return "NOK";  
        } else {  
            System.out.println("Data found! :-)");  
            System.out.println(jCtx.getTransientUserData() );  
            return "OK";  
        }  
    }  
}
```

BEISPIEL: PARTITIONED HELLO CHUNK

```
JobOperator jobOperator = BatchRuntime.getJobOperator();
long executionId = jobOperator.start("helloJob", null);

JobExecution jobExecution = jobOperator.getJobExecution(executionId);
System.out.println("Job Status: " + jobExecution.getBatchStatus());

// Wird nur im standalone Beispiel benoetigt
Batches.waitFor(executionId);

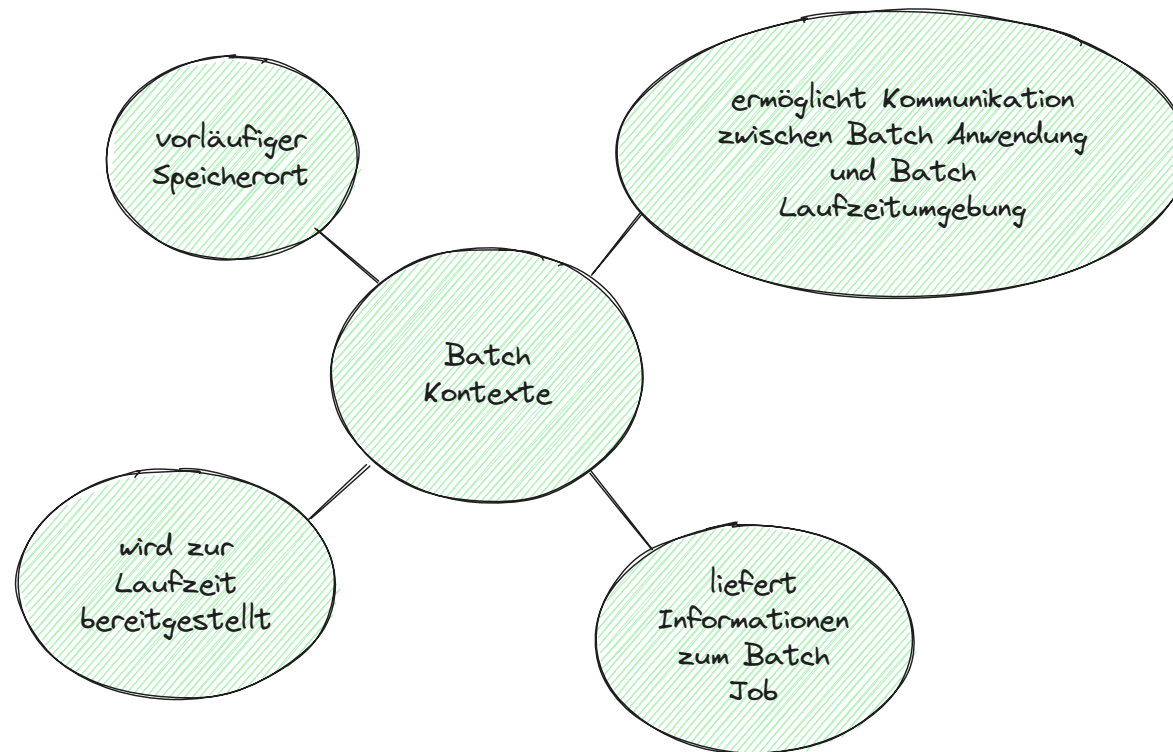
System.out.println("Exit Status: " + jobExecution.getExitStatus());
if(BatchStatus.COMPLETED == jobExecution.getBatchStatus()) {
    System.out.println("Batch job completed successfully.");
} else {
    System.out.println("Batch job failed with exit status: " +
        jobExecution.getExitStatus());
}
```

PARTITIONED CHUNK AUFGABE (TEIL 2)

- Erweiterung des vorherigen Beispiels
- Verlege die Erzeugung der Tabellen in entsprechende Methode eines **Reducer**
- Stelle die Kunden aus München sowie deren Anzahl per **Collector** bereit
- Nehme die Ergebnisse der Partitionen in einem Analyzer entgegen und mache sie über den **stepContext** für den Reducer zugänglich.
- Nehme die Ergebnisse in einem **Reducer** aus dem **StepContext** und gebe diese aus.

WEITERE INTERFACES

BATCH CONTEXTS



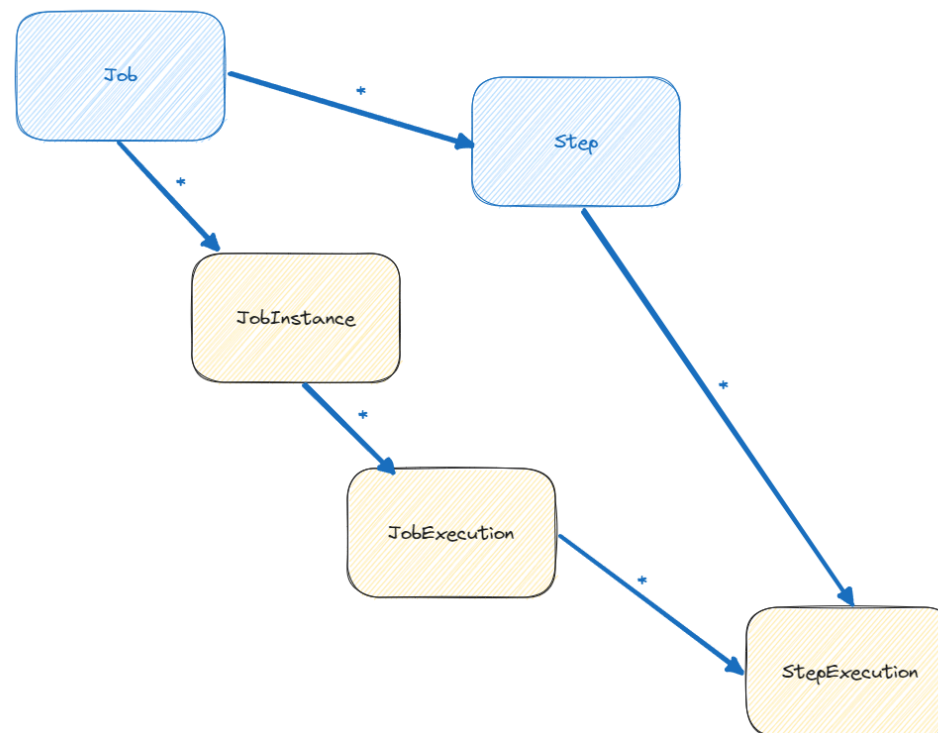
STEP CONTEXT

```
public interface StepContext {  
    public String getStepName();  
    public Object getTransientUserData();  
    public void setTransientUserData(Object data);  
    public long getStepExecutionId();  
    public Properties getProperties();  
    public Serializable getPersistentUserData();  
    public void setPersistentUserData(Serializable data);  
    public BatchStatus getBatchStatus();  
    public String getExitStatus();  
    public void setExitStatus(String status);  
    public Exception getException();  
    public Metric[] getMetrics();  
}
```

JOB CONTEXT

```
public interface JobContext {  
    public String getJobName();  
    public Object getTransientUserData();  
    public void setTransientUserData(Object data);  
    public Properties getProperties();  
    public BatchStatus getBatchStatus();  
    public String getExitStatus();  
    public void setExitStatus(String status);  
    public long getInstanceId();  
    public long getExecutionId();  
}
```

Jobs UND Steps ZUR LAUFZEIT



LISTENER

- Listener implementieren das **Interceptor** Pattern
- Es gibt Listener für verschiedene Artefakte
 - Job
 - Step (für Chunk- und Task-Steps)
 - Chunk (nur für Chunk-Steps)
 - Chunk Item (nur für Chunk-Steps)
 - Skip (nur für Chunk-Steps)
 - Retry (nur für Chunk-Steps)

CODEBEISPIEL: JobListener

```
package jakarta.batch.api.listener;  
  
public interface JobListener {  
    public void beforeJob() throws Exception;  
    public void afterJob() throws Exception;  
}
```

- Zur Vereinfachung der Implementierung existieren entsprechende abstrakte Basisklassen

CODEBEISPIEL: HelloJobListener

```
public class HelloJobListener implements JobListener {

    @Inject
    private JobContext jCtx;
    private long startTime;

    @Override
    public void beforeJob() {
        startTime = System.currentTimeMillis();
        System.out.println("Batch Job '" + jCtx.getJobName() + "' is starting...");
    }

    @Override
    public void afterJob() {
        long endTime = System.currentTimeMillis();
        System.out.println("Batch job ended after " + (endTime - startTime) + "ms.");
    }

}
```

CODEBEISPIEL: JobListener KONFIGURIEREN

```
<job id="jobID">
  <listeners>
    <listener ref="HelloJobListener"></listener>
  </listeners>
  ...
</job>
```

- Auch unter `chunk` und `batchlet` möglich, jeweils mit den entsprechenden Listener Typen.

BEISPIEL: LISTENERS ON RESTARTABLE CHUNK

```
<job id="helloJob" restartable="true">
  <listeners>
    <listener ref="de.mathema.batch.hello.listener.HelloJobListener" ></listener>
  </listeners>
  <step id="helloStep">
    <listeners>
      <listener ref="de.mathema.batch.hello.listener.HelloStepListener" ></listener>
      <listener ref="de.mathema.batch.hello.listener.HelloRetryListener" ></listener>
    </listeners>
    <chunk>
      ...
    </chunk>
  </step>
</job>
```

AUFGABE: Listeners

- Erweiterung der Aufgabe "Retryable Chunk"
- Messung der Laufzeiten des **Jobs** und der **Steps**
- Ausgabe der **Exceptions** im Falle eines **Retry** oder **Skip**

JOB SPECIFICATION LANGUAGE (JSL)

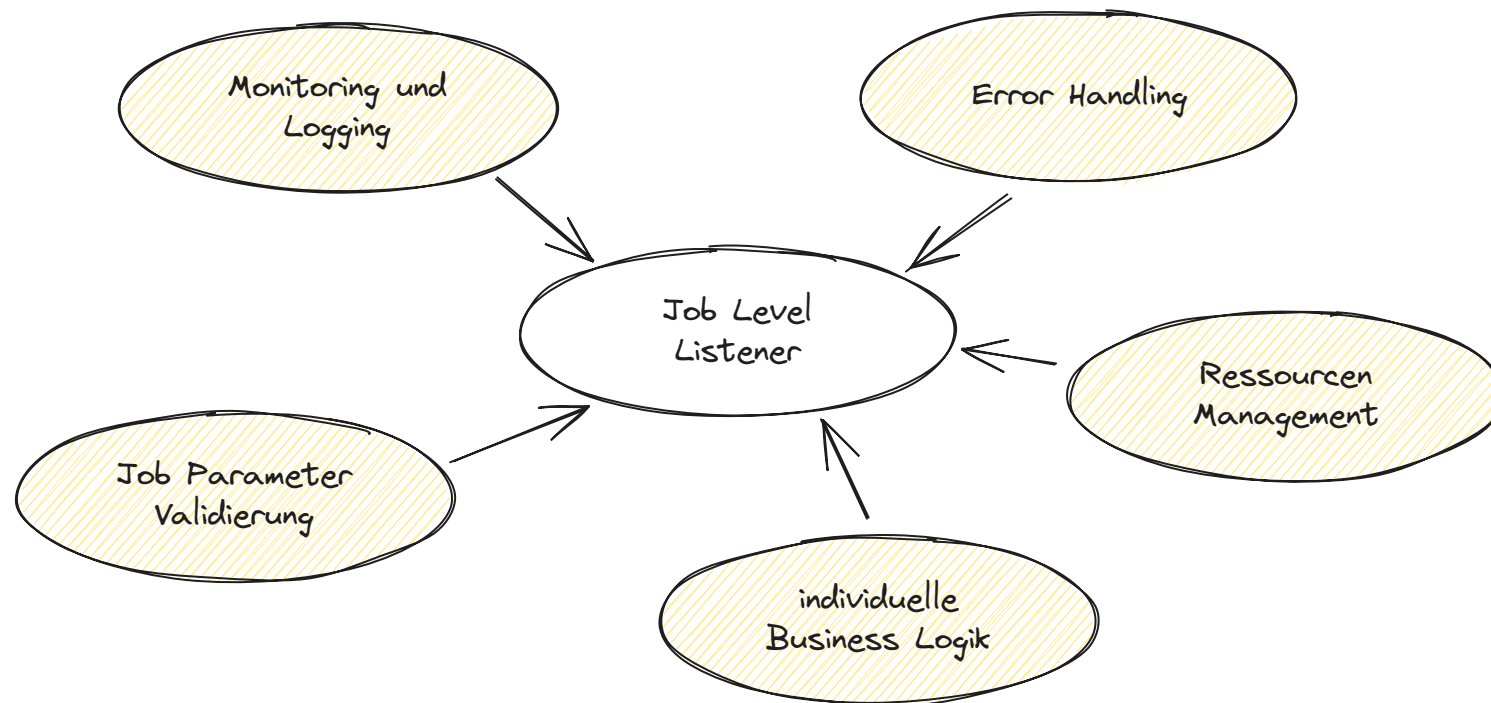
JOB SPECIFICATION LANGUAGE (JSL)

- spezifiziert einen Job, seine Schritte und steuert seine Ausführung
- JSL für Jakarta Batch ist in XML implementiert und wird daher auch als "Job XML" genannt

JOB SYNTAX

```
<job id="name" restartable="true|false" ></job>
```

JOB LEVEL LISTENER



JOB LEVEL LISTENER

- werden genutzt, um auf den Lifecycle eines Batch Jobs zu reagieren und ihn zu beobachten
- ist der einzige Listener, der auf Jobebene agiert
- mehrere Listener können auf einen Job konfiguriert werden
- geordnete Reihenfolge nicht garantiert

```
<listeners>
  <listener ref="name"></listener>
  ...
</listeners>
```

JOB LEVEL LISTENER PROPERTIES

- wird genutzt, um Property Werte an den Job Listener weiterzugeben
- es kann eine beliebige Anzahl an Properties gesetzt werden

```
<properties>  
  <property name="property-name" value="name-value"></property>  
</properties>
```

STEP SYNTAX

- Steps können nicht nur nach einem Step ausgeführt werden, sondern nach einem Flow, einer Decision oder einem Split
- besitzt auch Step Level Listener und Step Level Properties

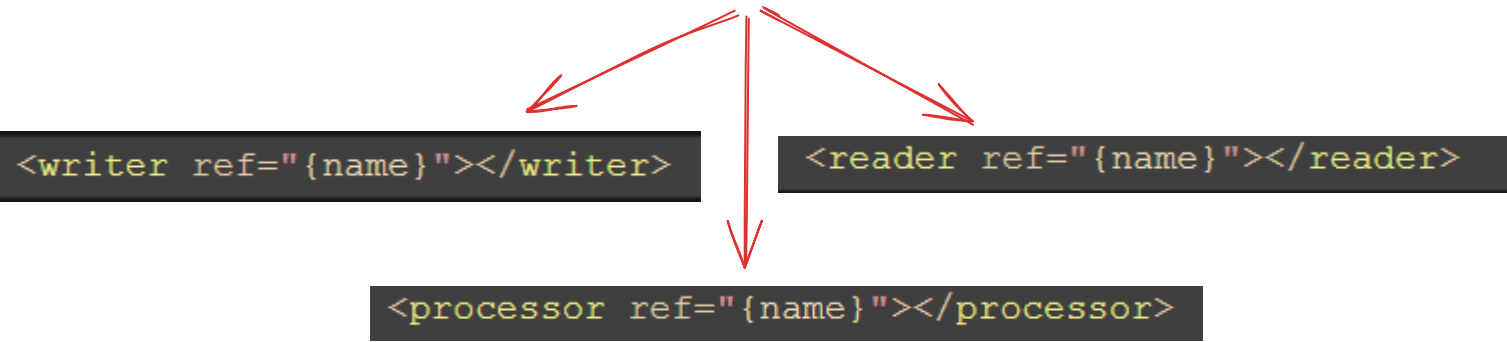
```
<step id="name"  
  start-limit="integer"  
  allow-start-if-complete="true|false"  
  next="flow-id|step-id|split-id|decision-id" ></step>
```

CHUNK STEP

```
<chunk checkpoint-policy="item|custom"  
  item-count="value"  
  time-limit="value"  
  skip-limit="value"  
  retry-limit="value"></chunk>
```

READER, PROCESSOR UND WRITER ELEMENT

- gibt den Namen des Batch Artifacts an



```
<writer ref="{name}"></writer>
```

```
<reader ref="{name}"></reader>
```

```
<processor ref="{name}"></processor>
```

READER, PROCESSOR UND WRITER PROPERTY

```
<properties>  
  <property name="property-name" value="name-value"></property>  
</properties>
```

CHUNK EXCEPTION HANDLING

- wenn während der Batch Laufzeit eine Exception geworfen wird, endet der Job mit dem Batch Status "FAILED"
- Reader, Processor und Writer können so konfiguriert werden, dass nach diesen Exceptions der Vorgang entweder neu gestartet oder übersprungen wird

SKIPPABLE EXCEPTION

```
<skippable-exception-classes>  
  <include class="java.lang.Exception"></include>  
  <exclude class="java.io.FileNotFoundException"></exclude>  
</skippable-exception-classes>
```

RETRYABLE EXCEPTION

```
<retryable-exception-classes>  
  <include class="java.io.IOException"></include>  
  <exclude class="java.io.FileNotFoundException"></exclude>  
</retryable-exception-classes>
```

NO-ROLLBACK EXCEPTION

```
<no-rollback-exception-classes>  
  <include class="class name"></include>  
  <exclude class="class name"></exclude>  
</no-rollback-exception-classes>
```

BATCHLET

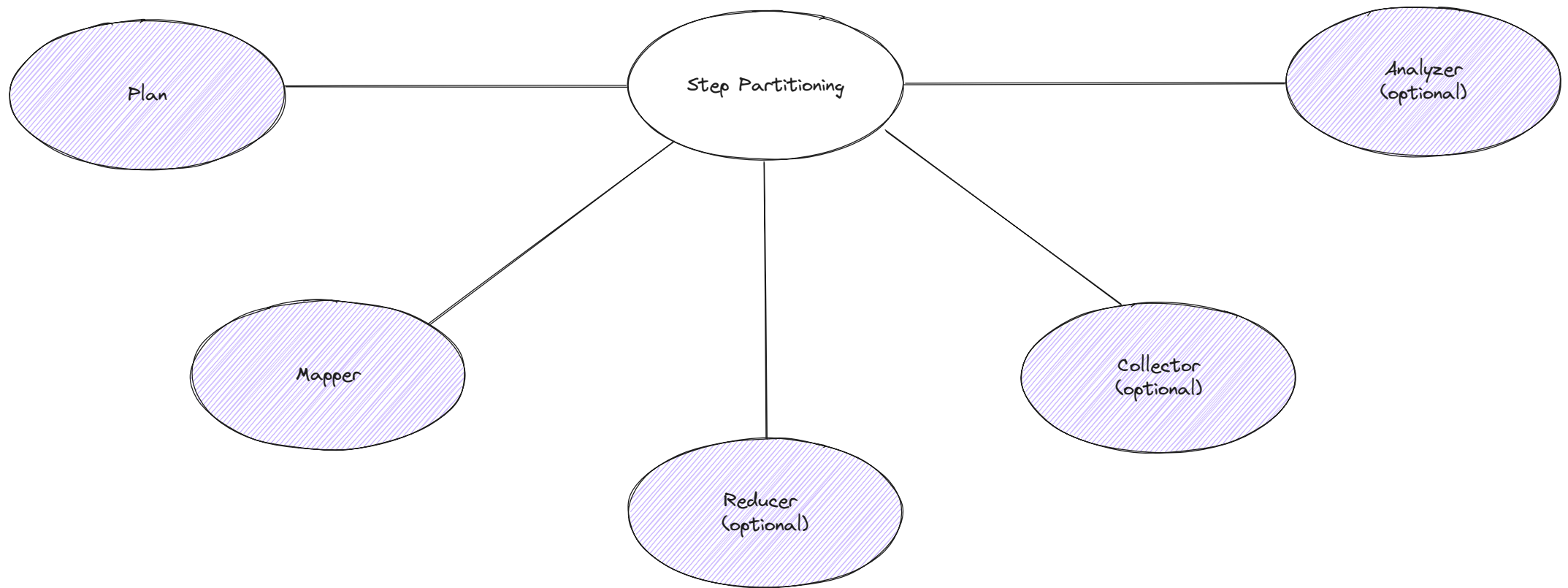
```
<batchlet ref="name"></batchlet>
```

STEP SEQUENCE

- falls die Reihenfolge der Steps wichtig ist, sollten sie sequenziell implementiert werden
- auch möglich für Flows, Splits oder Decisions

```
<next on="exit status" to="id" ></next>
```

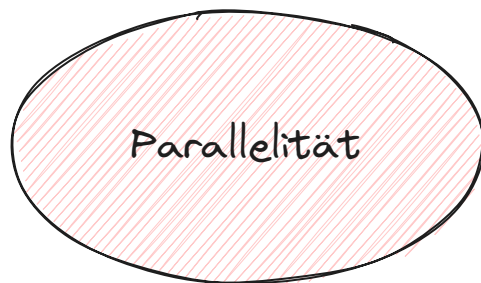
STEP PARTITIONING



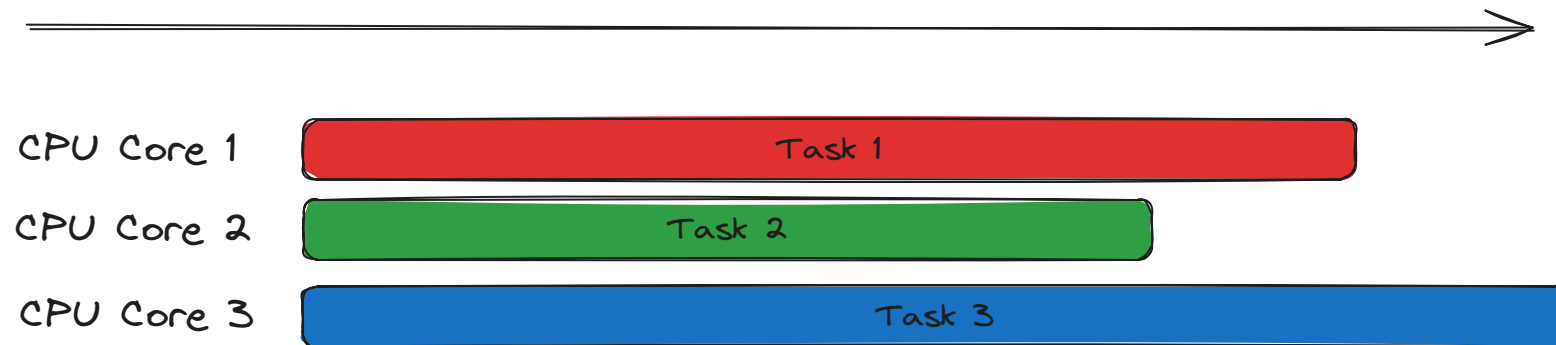
STEP PARTITIONING

```
<step id="Step1">  
  <chunk ...></chunk> or <batchlet ... ></batchlet>  
  <partition ...></partition>  
</step>
```

RÜCKBLICK: PARALLELITÄT



- mehrere Aufgaben werden zur selben Zeit bearbeitet



PARTITION PLAN

```
<step id="Step1">  
  <chunk ...></chunk>  
  <partition>  
    <plan partitions="3" threads="2"></plan>  
  </partition>  
</step>
```

PARTITION PROPERTY

```
<properties partition="{partition-number}">  
  <property name="{property-name}" value="{name-value}"></property>  
</properties>
```

PARTITION MAPPER

```
<partition>  
  <mapper ref="MyStepPartitioner"></mapper>  
</partition>
```

PARTITION COLLECTOR

```
<partition>  
  <collector ref="MyStepCollector"></collector>  
</partition>
```

PARTITION ANALYZER

```
<partition>  
  <analyzer ref="MyStepAnalyzer"></analyzer>  
</partition>
```

PARTITION REDUCER

```
<partition>  
  <reducer ref="MyStepPartitionReducer"></reducer>  
</partition>
```

FLOW

```
<flow id="{name}" next="{flow-id|step-id|split-id|decision-id}">  
  <step> ... </step>  
</flow>
```

DECISION

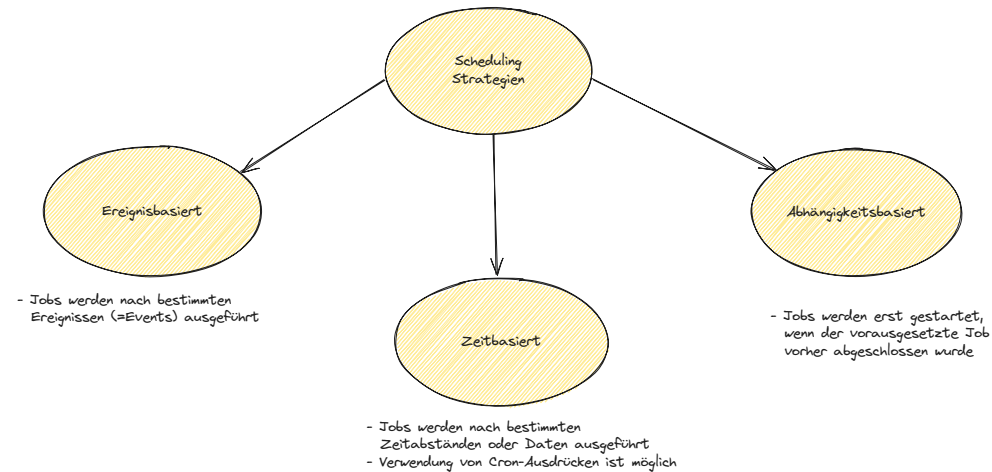
```
<decision id="{name}" ref="{ref-name}" ></decision>
```

SPLIT

```
<split id="{name}"next="{flow-id|step-id|split-id|decision-id}">  
  <flow> ... </flow>  
  ...  
</split>
```

JOB SCHEDULING

SCHEDULING STRATEGIEN

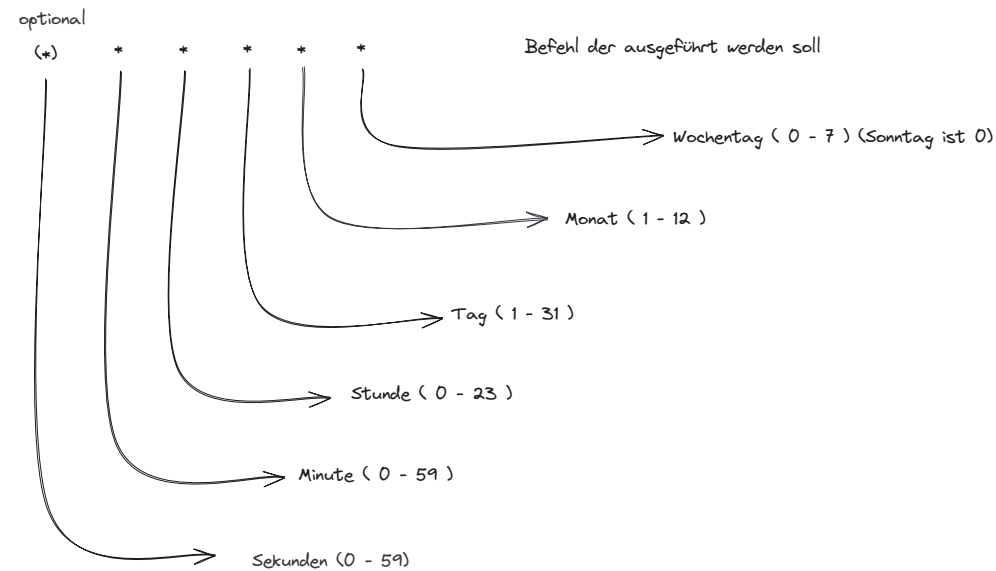


- Wann wird ein Batchjob gestartet?
- Verantwortung liegt AUSSERHALB der Batch API

ZEITBASIERTES SCHEDULING - CRON

- Der Cron Deamon ist ein zeitbasierter Job-Scheduler in Unix oder Unix-ähnlichen Betriebssystemen
- Möglichkeit zur Automatisierung von Befehlen und Shell-Skripten
- Syntax wurde als Quasi Standard in vielen Bereichen übernommen (z.B. als `@Schedule` in Jakarta Enterprise Beans oder Jakarta CDI)

CRON SYNTAX - ALLGEMEIN



- '*': Beliebiger Zeitpunkt
- '-': Bereichsangabe (z.B. 4–8)
- ',': Aufzählung (z.B. 15,30,45)
- '/': Intervallangabe (z.B. */5)

CRON SYNTAX - BEISPIELE

0	0	2	*	*	*	jeden Tag um 2 Uhr
0	30	20	*	*	1-5	von Montag bis Freitag täglich um 20:30 Uhr
0	*	*	15,30	*	*	jeden 15. und jeden 30. Tag im Monat
0	10-20	2,5	*	*	*	jede 10. und jede 20. Minute in jeder 2. und 5. Stunde
0	*/15	2-6	*	*	*	alle 15 Minuten von 2 bis 6 Uhr morgens
0	*	*	*	4-5	0,6	jeden Samstag und Sonntag im April und im Mai

EINSCHUB: WILDFLY



- Anwendungsserver nach dem Jakarta-EE-Standard
- Teil der JBoss Middleware Frameworks
- plattformunabhängig

BEISPIEL: CRON SYNTAX

BEISPIEL: APPLICATIONSCOPED SCHEDULER

```
@ApplicationScoped
@Singleton
public class Scheduler {
    @Schedule(second="0/15",minute="*",hour="*",dayOfWeek="Mon-Fri")
    public void scheduleBatchJob() {
        System.out.println("Scheduling something...");
        ...
    }
}
```

CRON AUFGABE:

- Konfigurieren Sie einen ApplicationScoped SingletonBean so, dass sie an Wochentage zwischen 8 und 17 Uhr alle 3 Minuten eine Nachricht ausgibt.

BATCH JOBS IM WILDFLY SERVER (CDI)

CONTEXTS AND DEPENDENCY INJECTION (CDI)

- Modernes Komponentenmodell
- Teil von Jakarta EE (früher: Java EE)
- Ermöglicht
 - Dependency Injection
 - Scoping
 - Eventing
 - Zugriff per Expression Language (EL)

CDI DEPLOYMENT

- Packaging als Java Archiv (JAR)
- Benötigt (meist leeres) META-INF/beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://jakarta.ee/xml/ns/jakartaee/beans_4.
      version="4.0"
      bean-discovery-mode="all">
</beans>
```

JAVA PERSISTENCE API (JPA)

- Standard Persistenzmodell / OR Mapper in Jakarta EE
- Persistierung von Java Objekten in relationalen Datenbanken (Entitäten)
- Mapping per Default oder Annotation
- Unterstützt
 - Dependent Objects
 - Entity Relationen
 - Vererbung
 - Abfragen mit eigener Query Language (JPQL)

JPA DEPLOYMENT

- Packaging im Archiv der nutzenden Anwendung (meistens JAR)
 - Verarbeitungslogik meist als CDI Bean bzw. EJB
- Konfiguration via **META-INF/persistence.xml**
 - Ermöglicht automatische Tabellenanlage

CODEBEISPIEL: PERSISTENCE.XML (IM WILDFLY)

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
  <persistence-unit name="customer">
    <jta-data-source>java:jboss/datasources/ExampleDS</jta-data-source>
    <properties>
      <property name="hibernate.hbm2ddl.auto" value="create-drop"></property>
      <property name="hibernate.show_sql" value="true"></property>
    </properties>
  </persistence-unit>
</persistence>
```

CODEBEISPIEL: EINFACHE ENTITÄT

```
@Entity
public class Person implements Serializable {

    @Id
    private String id;
    private String vorname;
    private String nachname;

    public Person(String id, String vorname, String nachname) {
        this.id = id;
        this.vorname = vorname;
        this.nachname = nachname;
    }

    // Benötigt JPA
    private Person() {}

    // Getter und Setter nach fachl. Bedarf
}
```

CODEBEISPIEL: ARBEITEN MIT ENTITÄTEN

```
@Dependent
@Stateless
public class PersonDAOImpl implements PersonDAO {

    @PersistenceContext
    private EntityManager em;

    public String erzeugePerson(String vorname, String nachname) {
        Person p = new Person(UUID.randomUUID().toString(), vorname, nachname);
        em.persist(p);
        return p.getId();
    }

    public Person findePerson(String id) { return em.find(Person.class, id); }

    public Person aktualisierePerson(String id, String vorname, String nachname) {
        Person p = findePerson(id);
        if(p!=null) {
            p.setVorname(vorname);
            p.setNachname(nachname);
        }
        return p;
    }
}
```

BEISPIEL: JPA IM WILDFLY

```
public class PersonClient {  
  
    public static void main(String[] args) throws Exception {  
  
        InitialContext context = new InitialContext();  
        PersonDAO dao = (PersonDAO)context.lookup(  
            "hello-jpa/PersonDAOImpl!entity.PersonDAO"  
        );  
  
        String id = dao.erzeugePerson("Max", "Mustermann");  
  
        Person p = dao.findePerson(id);  
        System.out.println("Gefunden: " + p);  
  
        p = dao.aktualisierePerson(id, "Paula", "Musterfrau");  
        System.out.println("Daten aktualisiert: " + p);  
  
        p = dao.findePerson(id);  
        System.out.println("Gefunden: " + p);  
    }  
}
```

AUFGABE: CUSTOMER VIA JPA PERSISTIEREN

- Klasse `Customer` als `Entity` annotieren
- `DataAccessObject` für `Customer` als `Stateless Bean` implementieren.
 - Erzeugen von `Customer`
 - Suchen von `Customer` nach ID
 - Aktualisieren von `Customer`

BATCH JOB ARTEFAKTE IN CDI

```
@Named
public class HelloProcessor implements ItemProcessor {

    @Override
    public Object processItem(Object item) {
        System.out.println("Processing " + item + "...");
        return String.format("Hello %s!", item);
    }
}
```

- Sind i.d.R. **@Dependent** scoped Beans (Default Scope)
- Sollten **@Named** sein (zur Referenzierung im XML)

BATCH DEFINITION VIA BEAN NAMES

```
<job>
  ...
  <step id="readProcessedHelloData">
    <batchlet ref="helloBatchlet" ></batchlet>
  </step>
</job>
```

- ACHTUNG: Namen beginnen mit einem Kleinbuchstaben!

BEISPIEL: CDIBasedParallelChunk

```
@ApplicationScoped
@Singleton
public class MainApp {

    @Inject
    private JobOperator jobOperator;

    @Schedule(second="0/10",minute="*",hour="*")
    public void scheduleBatchJob() {
        long executionId = jobOperator.start("helloJob", null);

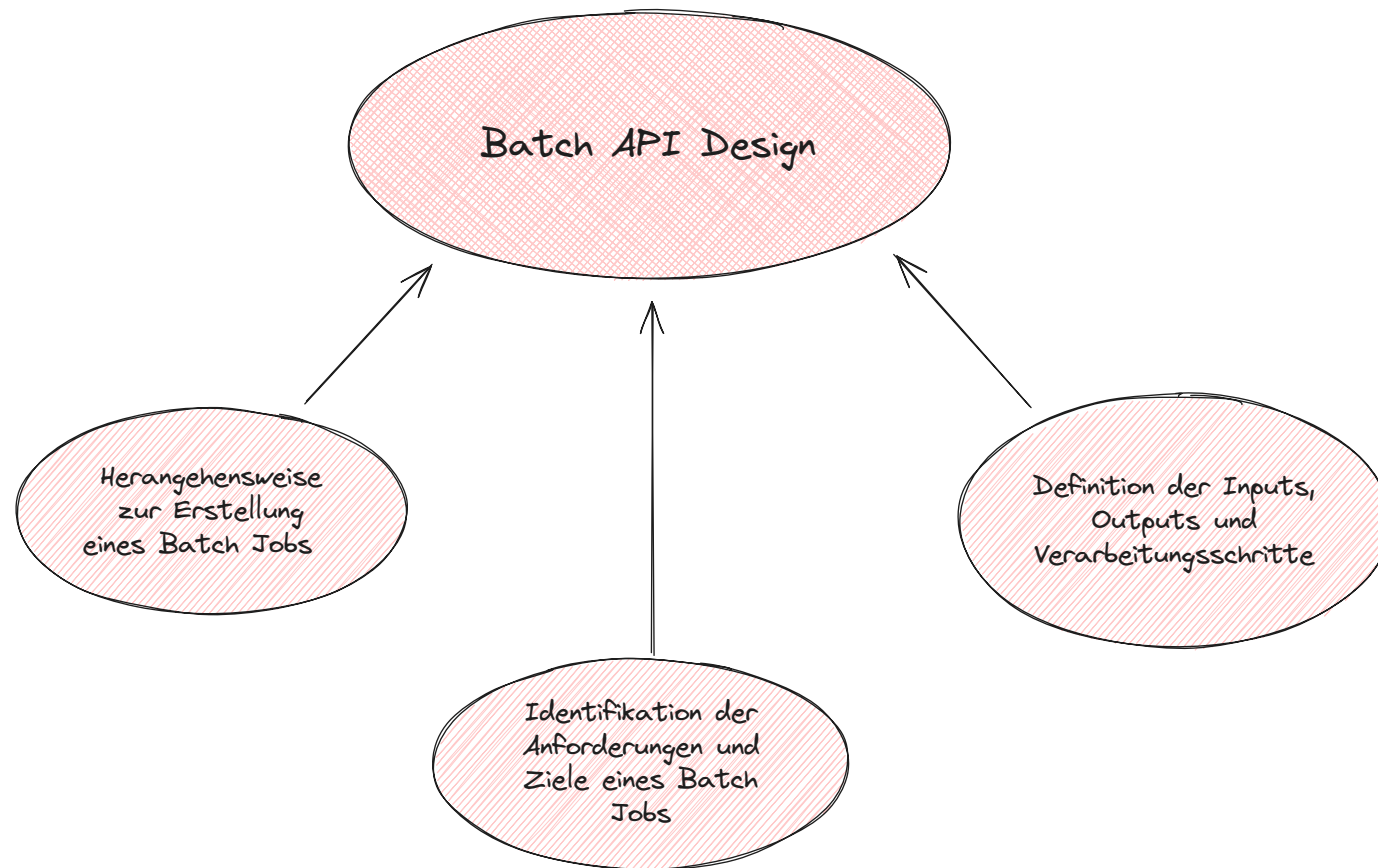
        JobExecution jobExecution = jobOperator.getJobExecution(executionId);
        System.out.println("Job Status: " + jobExecution.getBatchStatus());
    }
}
```

AUFGABE: WILDFLY SERVER MIT CDI

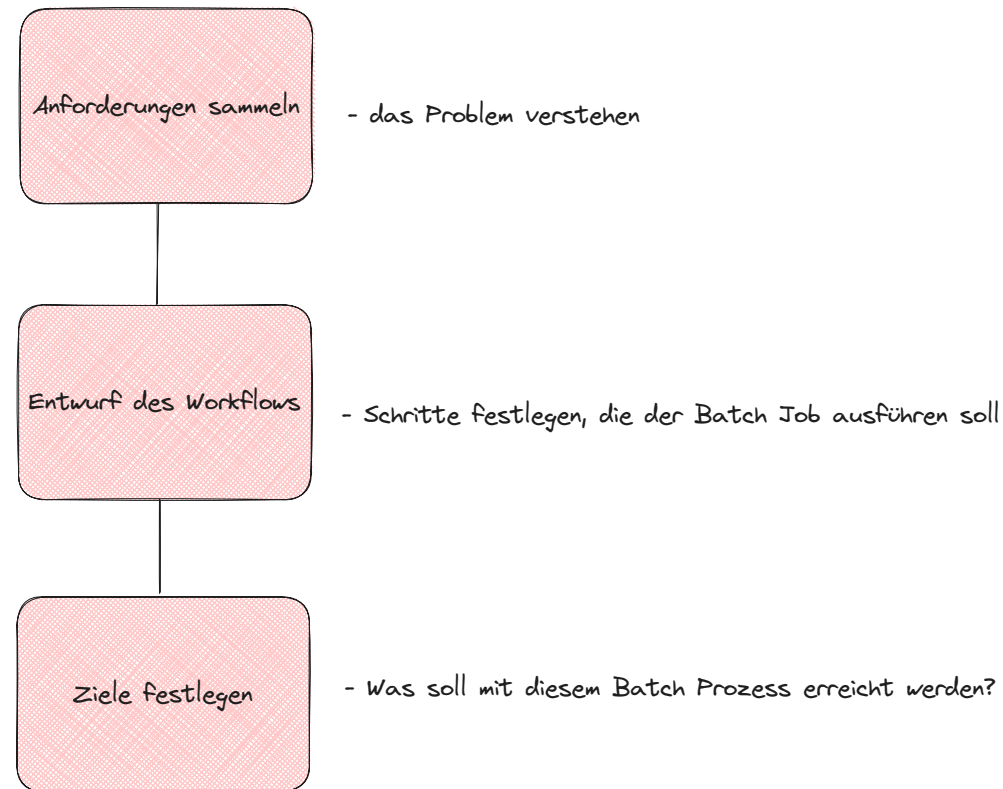
- Chunk Batchverarbeitung mit customer.csv möglich machen
- Speicherung des Kunden via JPA
- auf Wildfly deployen
- über H2Console auf Datenbank zugreifen

ANHANG BATCH JOB DESIGN

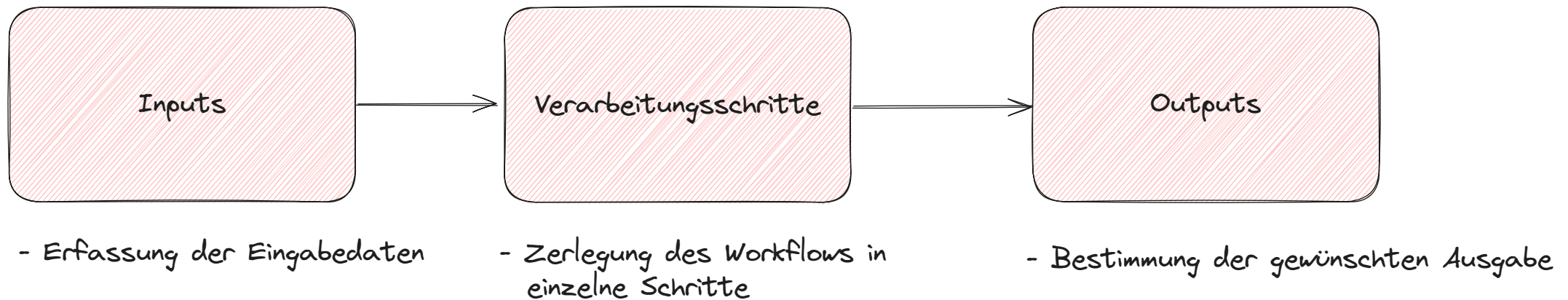
BATCH JOB DESIGN



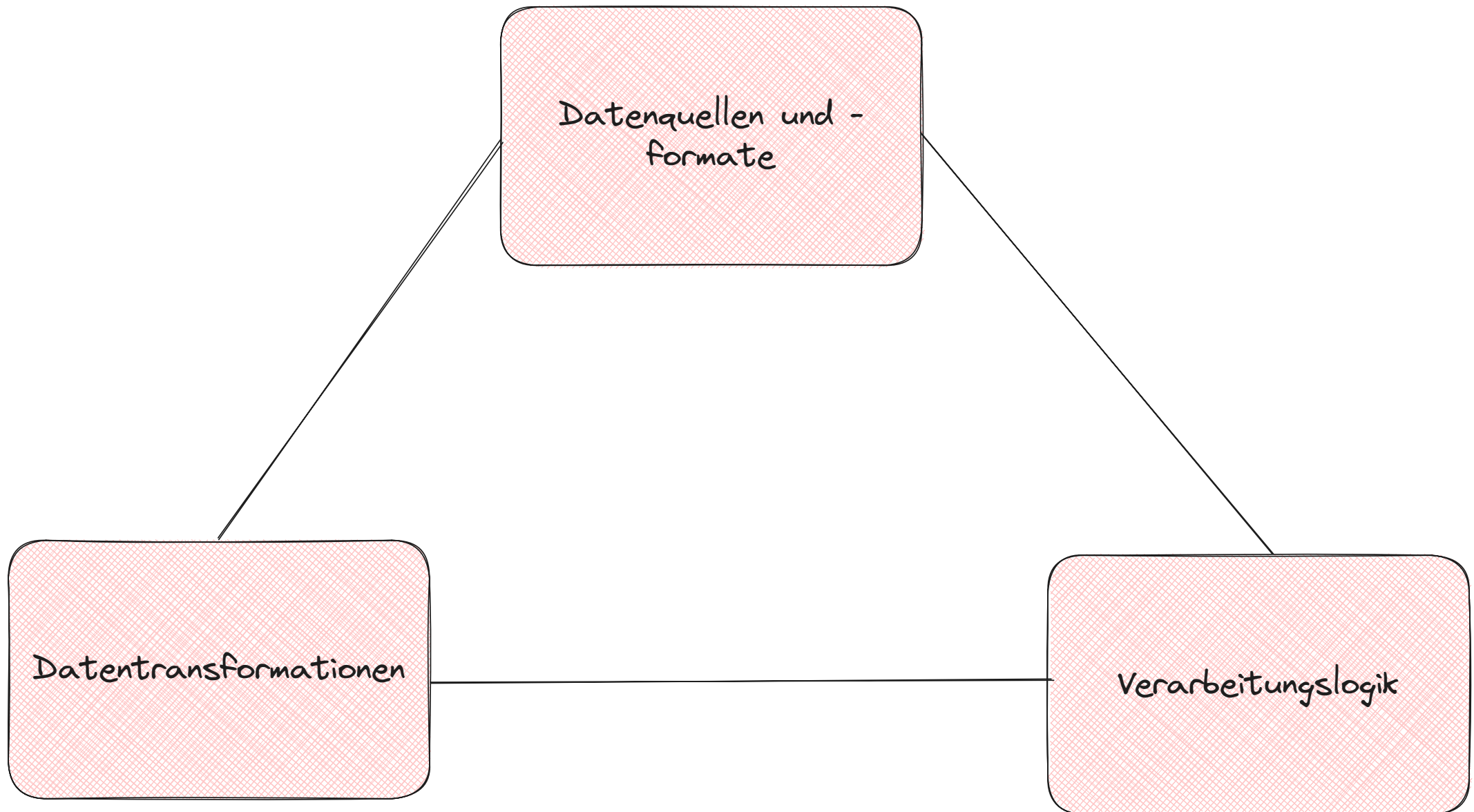
HERANGEHENSWEISE ZUR ERSTELLUNG EINES BATCH JOBS



DEFINITION VON INPUTS, VERARBEITUNGSSCHRITTEN UND OUTPUTS



IDENTIFIKATION DER ANFORDERUNGEN UND ZIELE EINES BATCH JOBS



Quellenangaben:

- <https://jakarta.ee/specifications/batch/2.1/>
- <https://docs.wildfly.org/34/>